ROSE5730 Autonomous Mobile Robots

Lecture 6: Motion Planning

Dongkun Han
Department of Mechanical and Automation Engineering;
The Chinese University of Hong Kong.



Outline

Motion planning

4 Sampling-based planning

3 Hierarchical planning

2 Graph-based planning

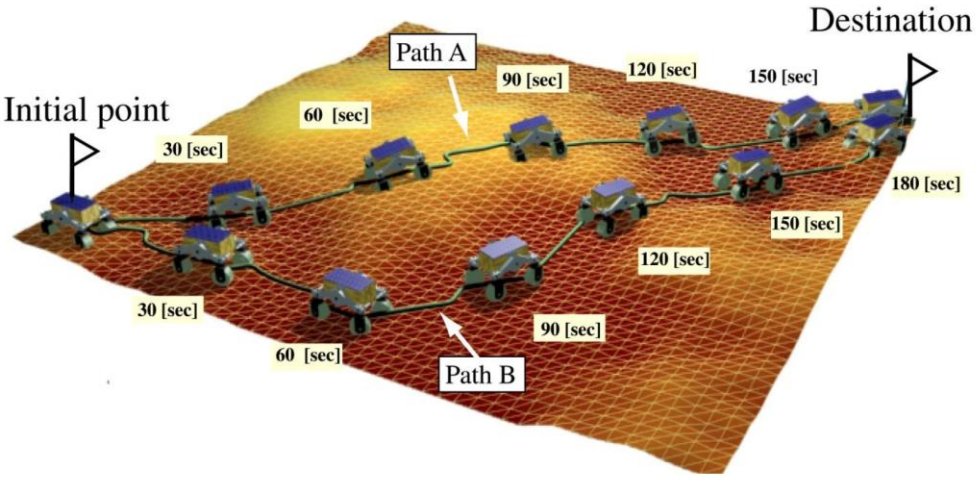
1 Representation



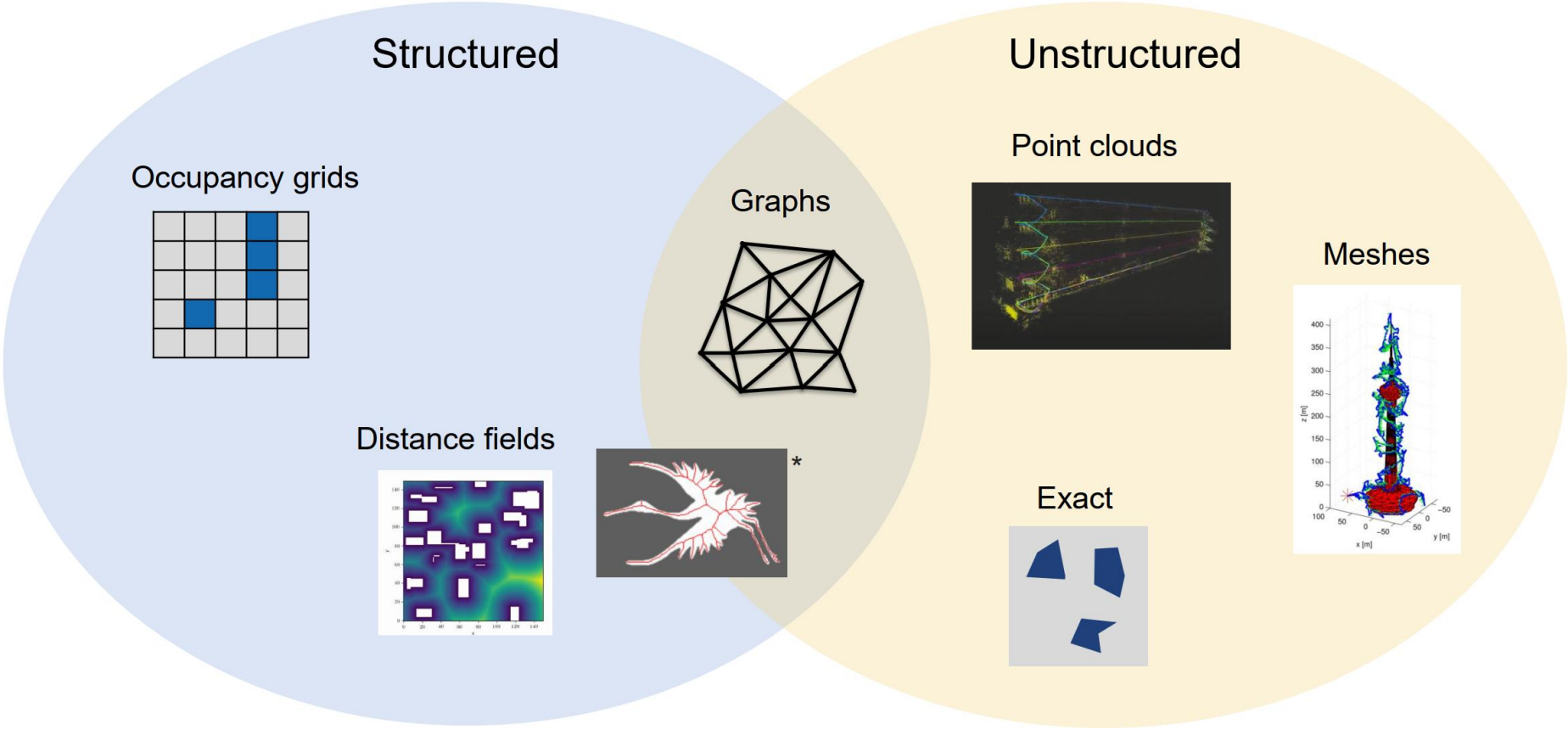
Motion planning: overview

- In a nutshell, work out how the robot could *feasibly* move from one position to another
  - Our **representation** of the robot and the world is sufficiently expressive
  - We know **where we are** and **where we want to go**
  - We have a **motion model** for our robot

- Typically cast as an optimization problem – minimize cost (time, distance, energy), within constraints



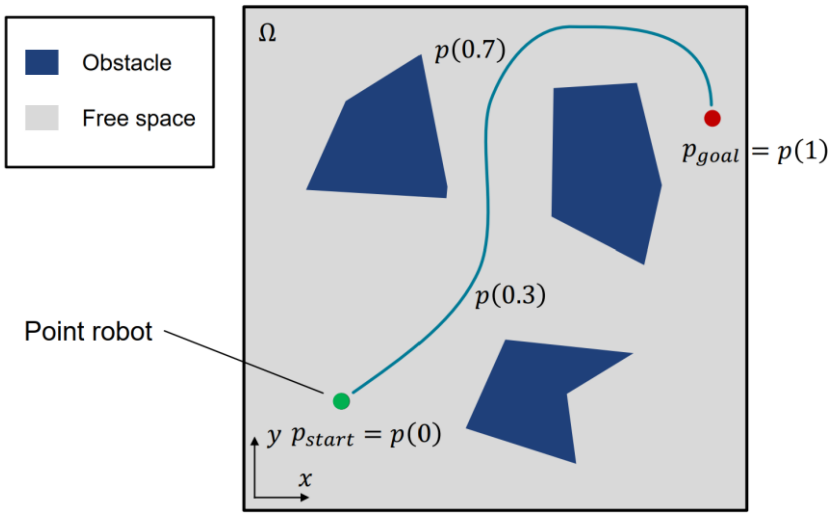
Representation



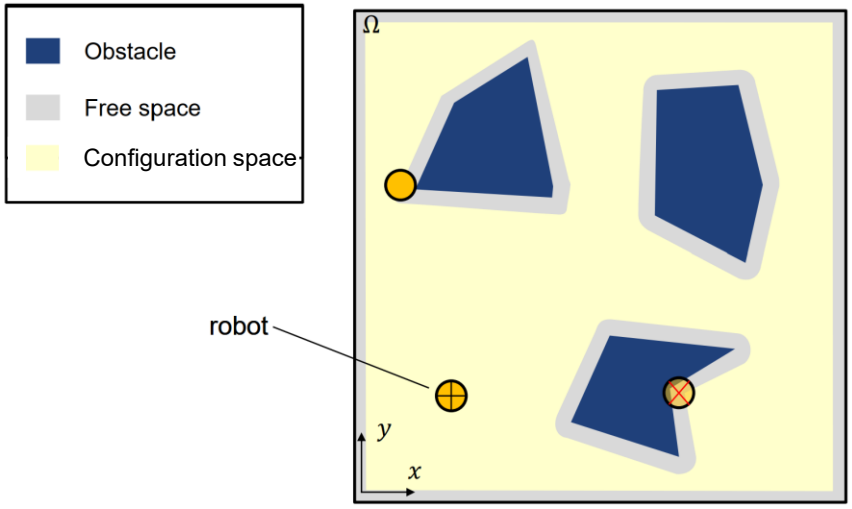
Chang, Sukmoon. "Extracting skeletons from distance maps." International Journal of Computer Science and Network Security 7.7 (2007): 213-219.

Representation

- **Workspace** is often the representation of the world, possibly independent of the robot itself. Often describes some notion of reachability, what space is free or occupied?
- **Configuration space** describes the full state of the robot in the world (actuator positions, orientation, etc.).



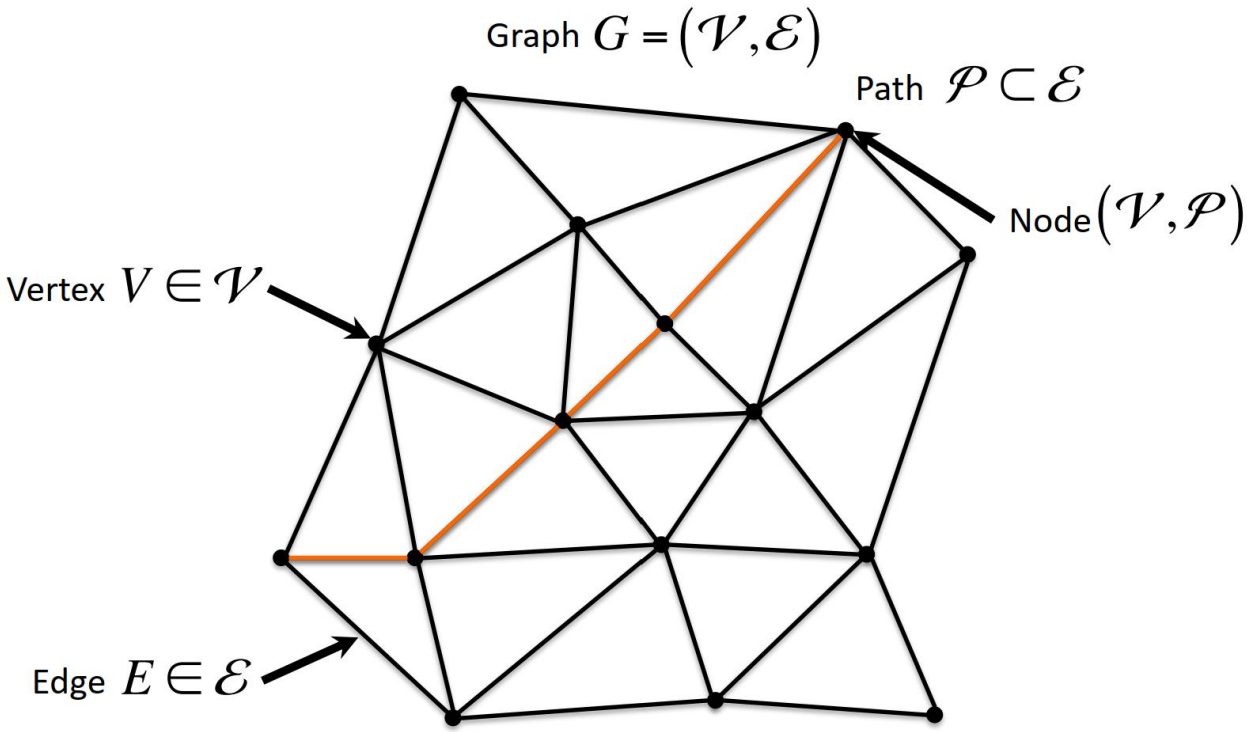
workspace and paths



configuration space

Representation

- Some Terminology

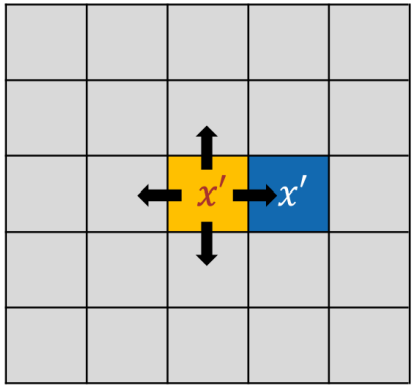


Directed graph: edges have direction
Weighted graph: edges have costs

Discrete state space representation

- Reduce continuous state space to a finite set of discrete **states**.

$$x \in X$$



- Also define feasible **actions** from each state

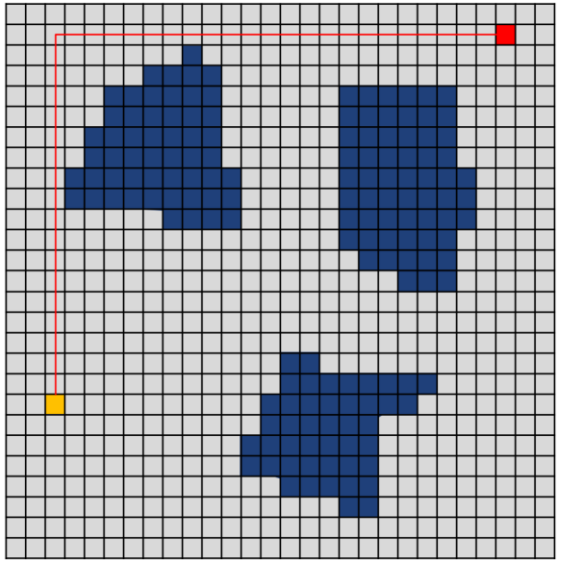
$$A(x) = \{a_0, a_1, \ldots, a_n\}$$

- And an associated **transition function**

$$f(x, a) = x'$$

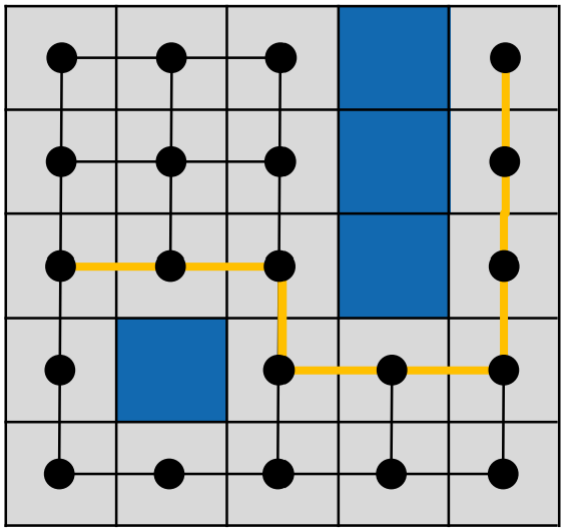
Issues of grid-based representations

- Usually suffer some **loss of precision**
- Selecting an appropriate grid resolution can be a challenge (multi-resolution mapping)
- Can limit the type of output path
- Suffer from poor scaling in higher dimensions



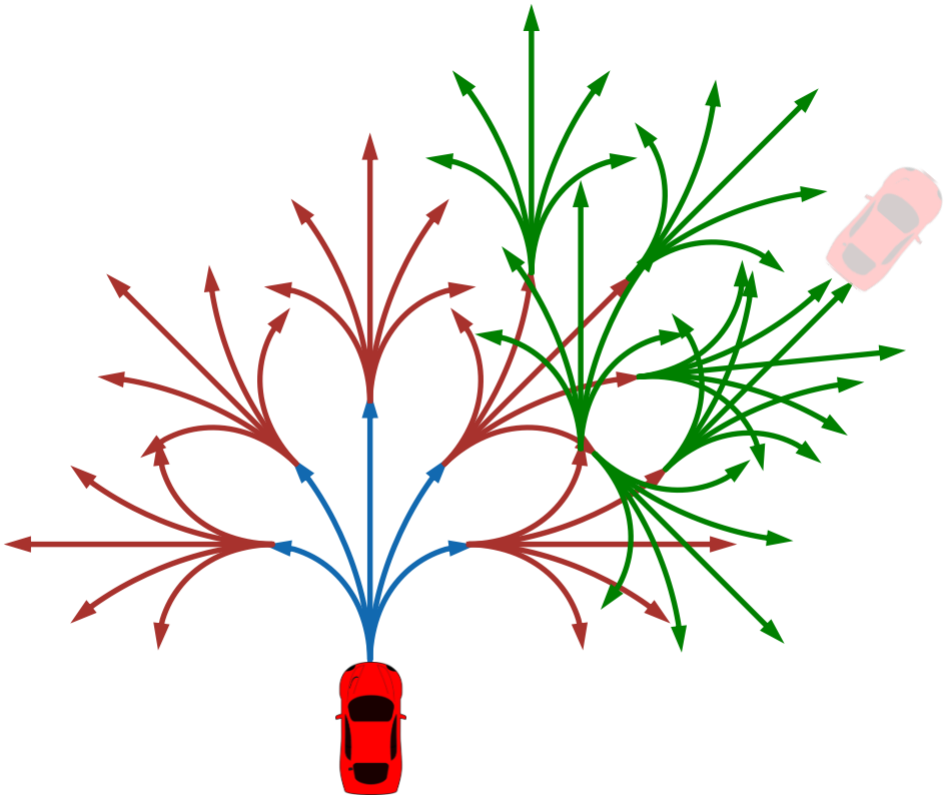
Grid → Graph

- Consider:
  - States as vertices
  - Transitions as directed edges
- The result is a graph
- Consider:
  - Start node, x_s
  - Global node, x_g
  - Cost function C: $X \times A \rightarrow R^+$
- Finding the shortest path can be treated as a graph search problem



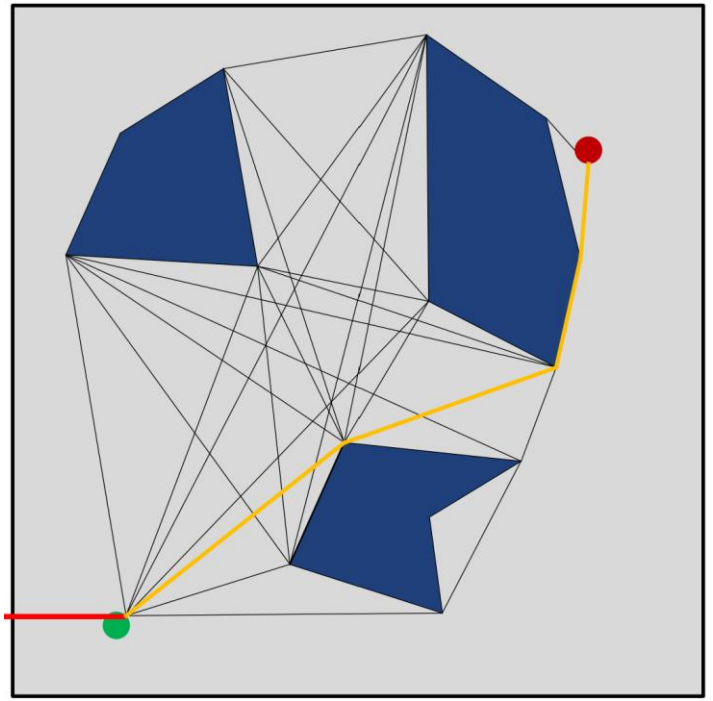
Other graph-based representations

- **Grid lattice** – create a set of feasible motion primitives, and construct a tree (graph) that chains the motions into a sequence (plan)



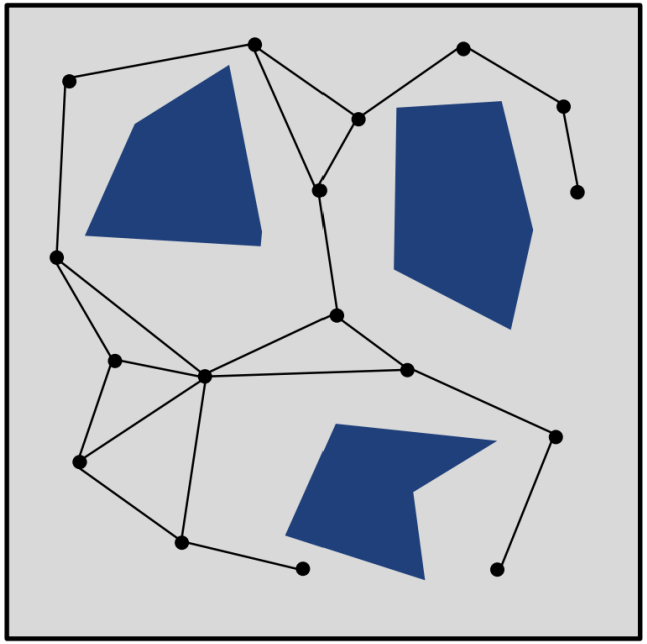
Visibility graph

- Create edges between all pairs of mutually visible vertices
- Search resulting graph
- Optimal plan!
- Limited to straight motion, 2D, polygonal obstacles



Randomly-sampled graphs

- Especially popular for sample-based methods
- Require careful consideration to construct graphs with guarantees

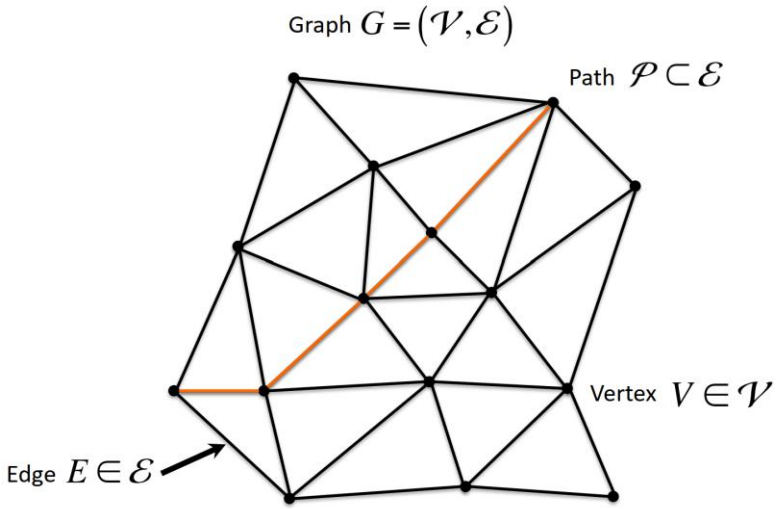


Planning with graphs

- Given a representation, a start, a goal, and a motion model, how do we actually generate a plan?
- Many planning approaches use graphs, because we know how to search graphs
- Solve your planning problem in three easy steps:
  - Convert problem to a graph
  - Search the graph
  - Profit!

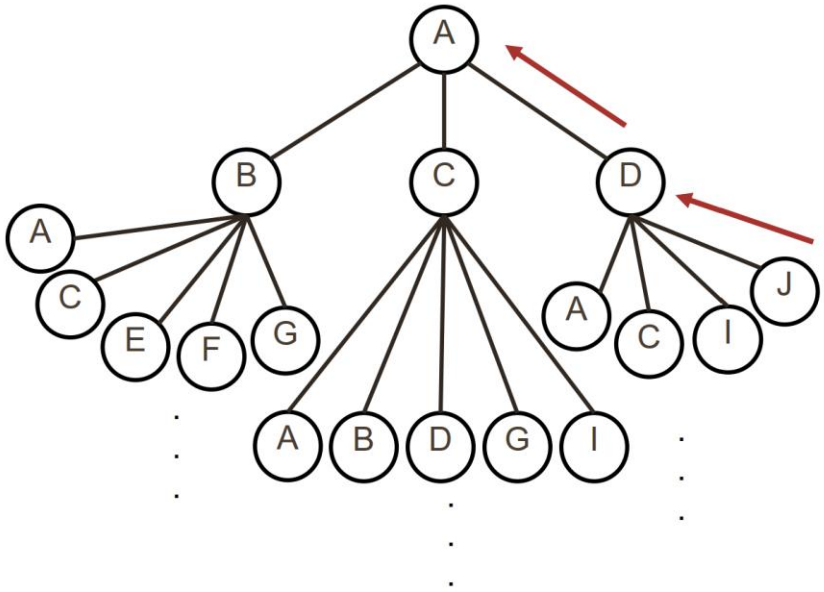
Graph search methods

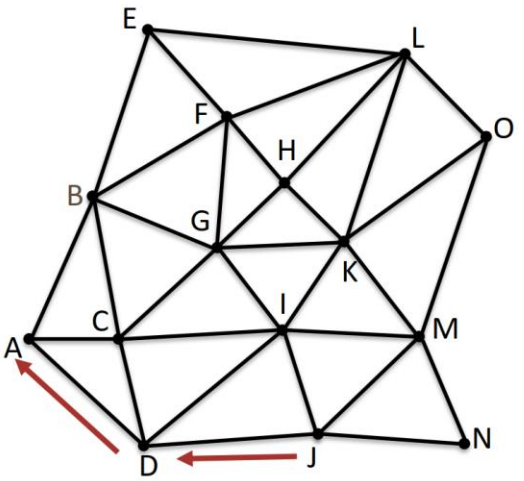
- Edges can also have associated weights
- We generally only consider positive edge weights
- A minimum cost path never revisits a node
- **Graph search methods:**
  - *Breadth-first search (BFS)*
  - *Depth-first search (DFS)*
  - *Dijkstra's Algorithm*
  - *A**



Search tree

- **A search tree:**
  - Start state at the root node
  - Children correspond to successors
  - A node corresponds to a unique plan from start to that state (follow up tree from node)
  - For most problems, we want to avoid building the whole tree
  - Use search algorithms to efficiently traverse tree

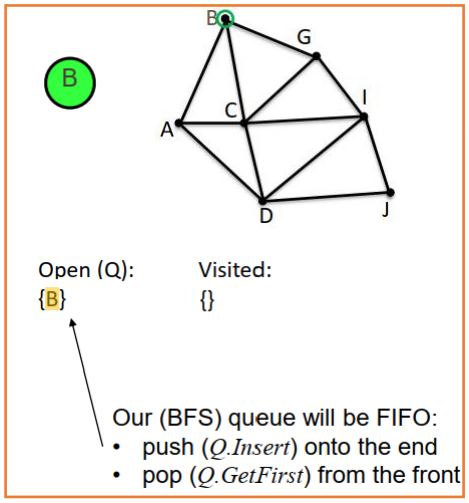


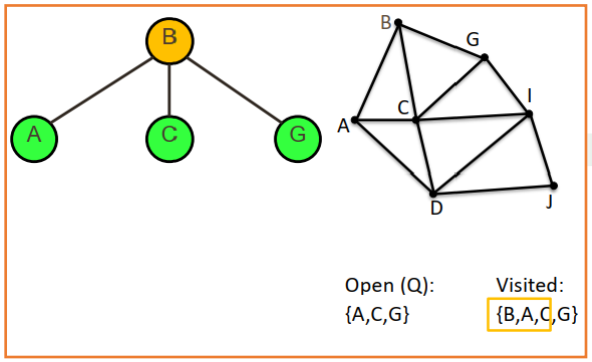


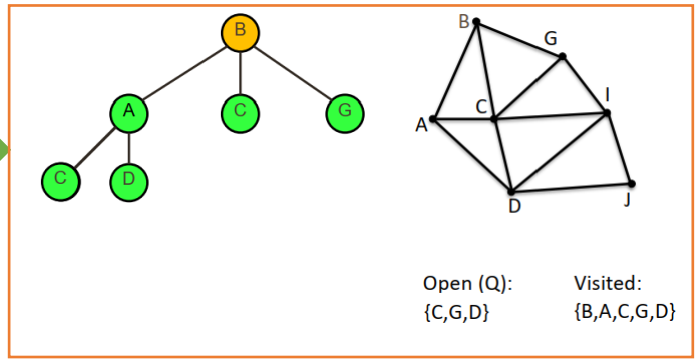
Basics of forward search

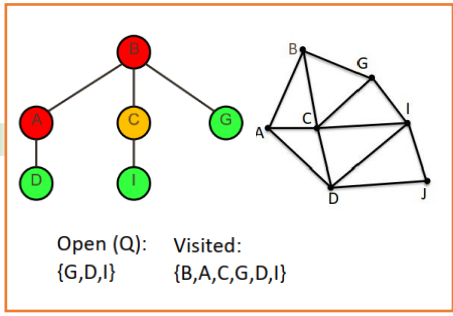
- Generally, start from the start, grow tree until you find a solution (path to goal)
- Expanding a node refers to adding children to the tree, pushing them onto the open set
- Try to expand as few tree nodes as possible
- **Open** set maintains a list of frontier (unexpanded) plans
  - Keeps track of what nodes to expand next
  - Often stored as a priority queue
  - For each node in the open list, we know of at least one path to it from the start
- **Closed** set keeps track of nodes that have been expanded
  - Convert for each node in the closed list, we've already found the lowest-cost path to it from the start

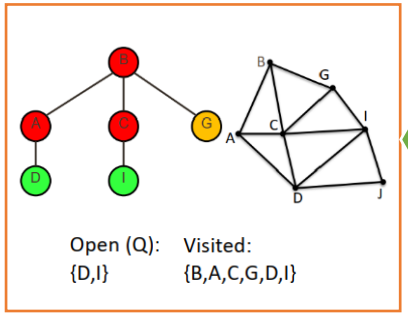
Example – BFS (Breadth-First Search)

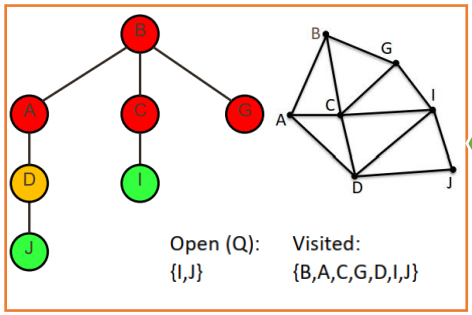


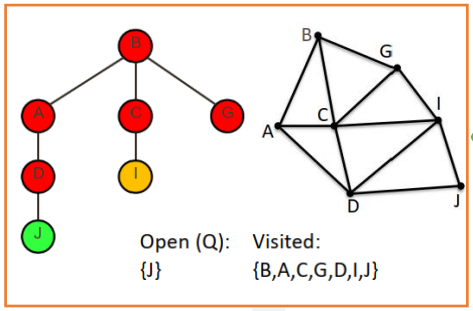


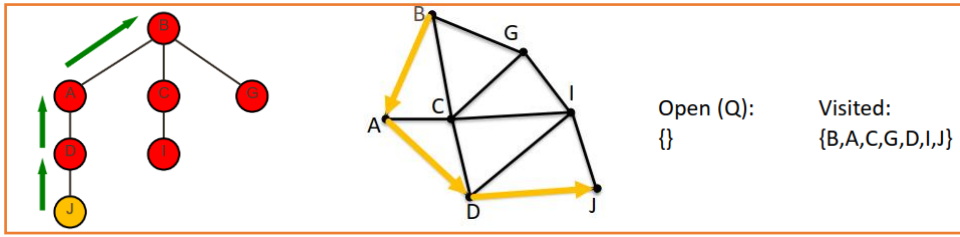








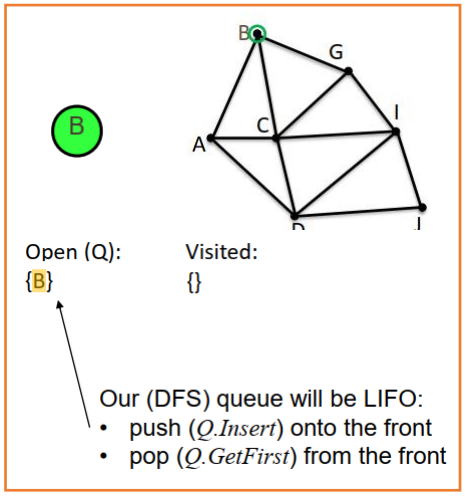


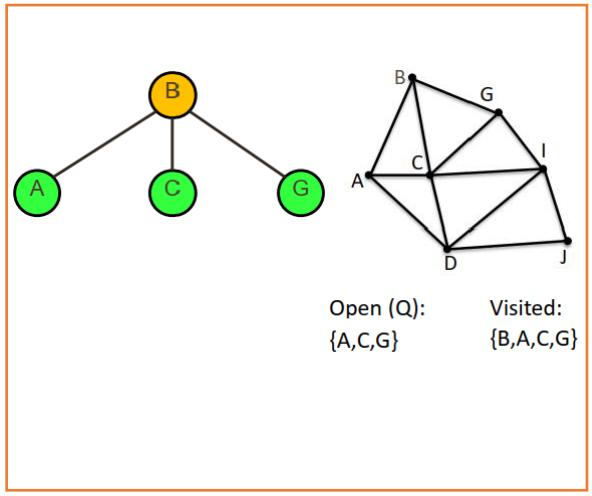


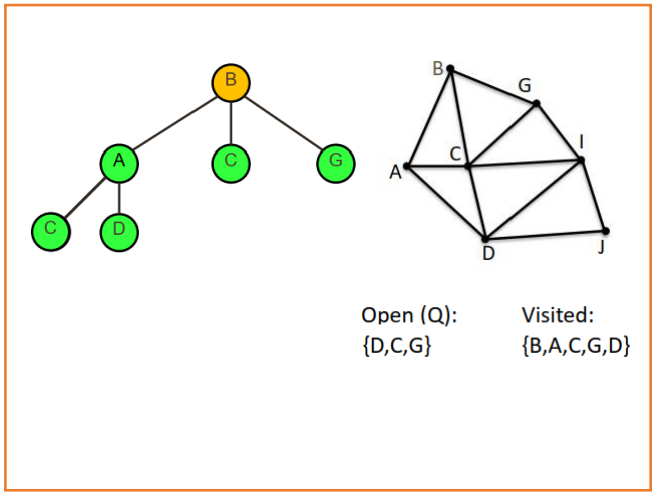
Final path solution: B→A → D → J

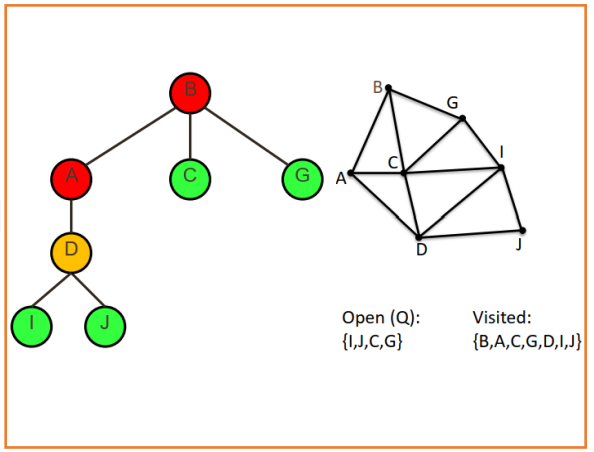
Other solutions may exist but have the same number or more transitions

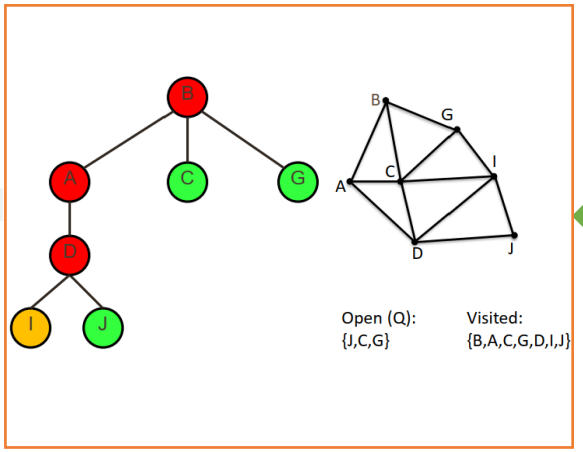
Example – DFS (Depth-First Search)

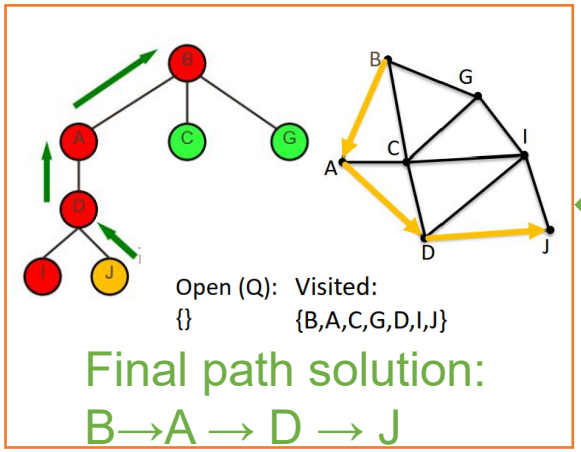












Final path solution:
B→A → D → J

Search tree comparison

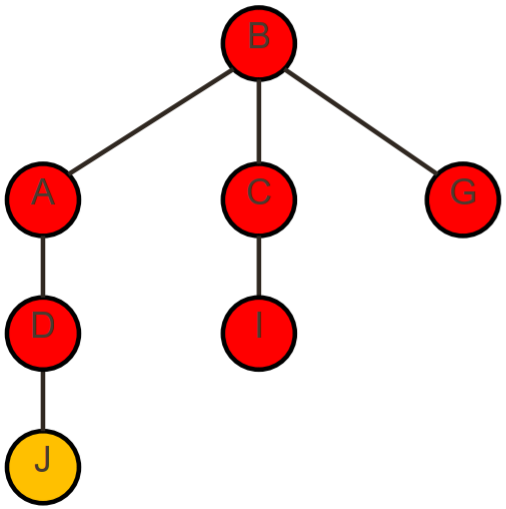
- Complete (will find the solution if it exists)
- Guaranteed to find the shortest (number of edges) path (First solution found is the optimal path)

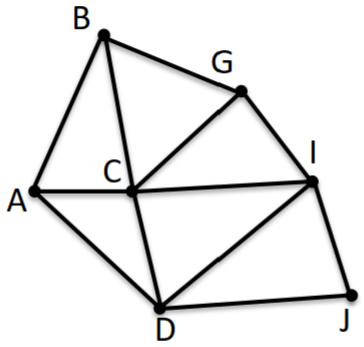
BFS

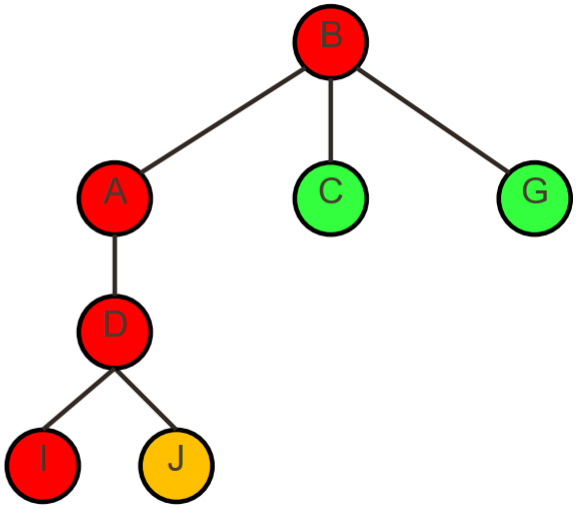
- Lower memory footprint than BFS with high-branching
- Not often used for path search, sometimes used to completely explore a graph
- Both BFS and DFS are simple to implement but might be inefficient. More complex algorithms are faster, but generally more difficult to implement
- Seems like we want a compromise, search promising paths while we can, then go back up if they aren't working out
- DFS not complete for infinite trees (may explore an incorrect branch infinitely deep, never come back up, BFS is complete)

DFS

Search tree comparison







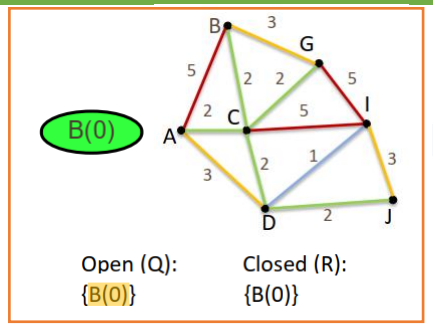
BFS

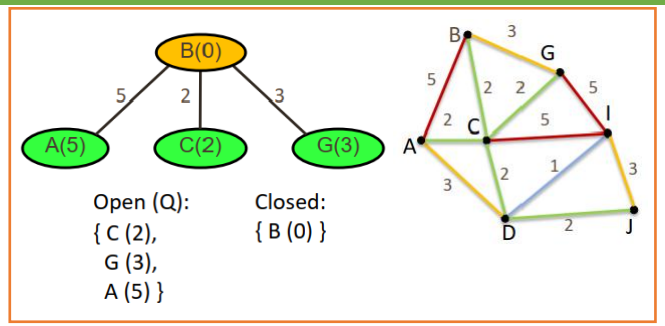
DFS

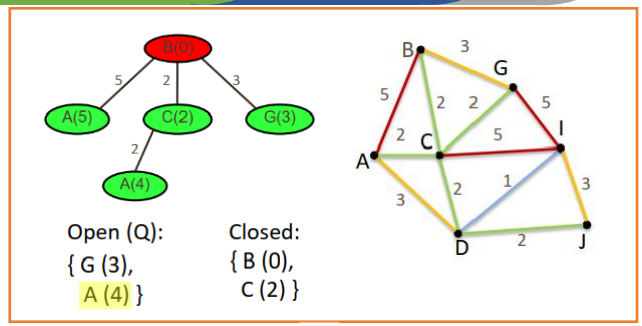
Dijkstra's algorithm

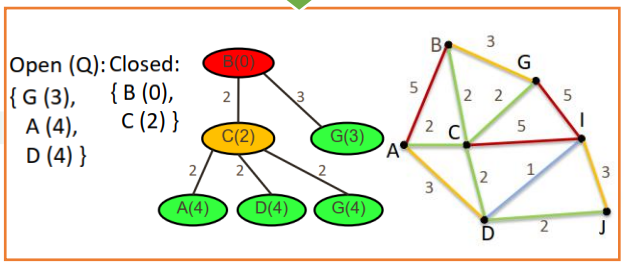
- Basic idea of expanding in order of closest to start (BFS with **edge costs**)
- One of the most commonly used routing algorithms in graph traversal problems
- Asymptotically the fastest known single-source shortest path algorithm for arbitrary directed graphs
- **Open** queue is ordered according to currently known best cost to arrive

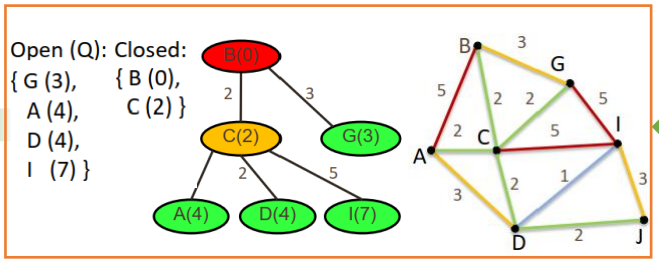
Example - Dijkstra's algorithm

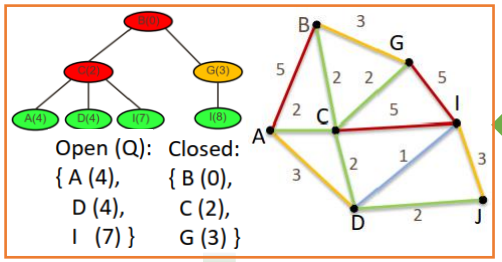


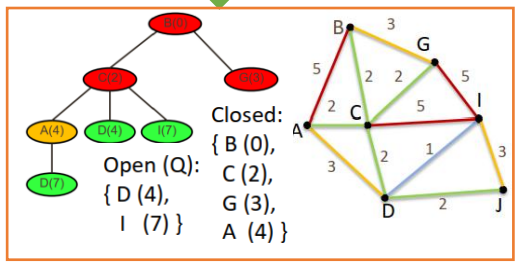


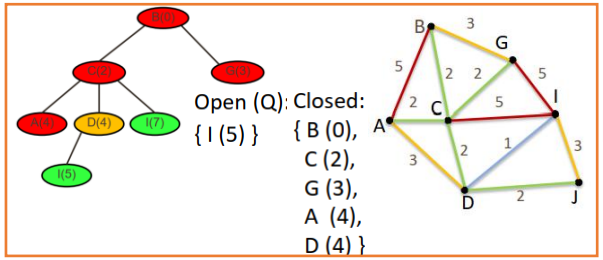


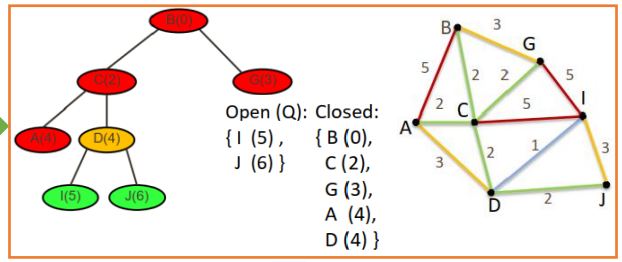


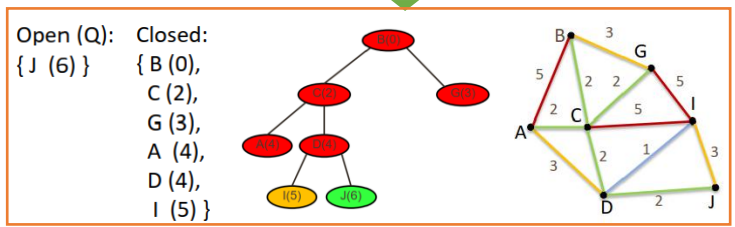


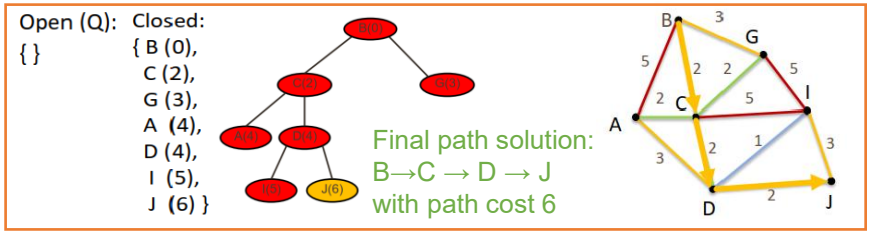












Dijkstra's algorithm

- At the end, we can recover the lowest-cost route from the start to any node (or any node with cost < goal if we terminate at a goal)
- Quite easy to implement, but requires a little bit of careful management with the priority queue
- Doesn't really know the goal exists until it reaches it
  - Could we guide the search to expand nodes that are closer to the goal earlier?
  - Can we do it without breaking the condition that a node is only accepted with its lowest cost of arrival?

A* heuristic search

- Heuristic:
  - Any optimistic estimate of how close a state is to a goal
  - Designed for a particular search problem
  - Examples: *Manhattan distance*, *Euclidean distance*

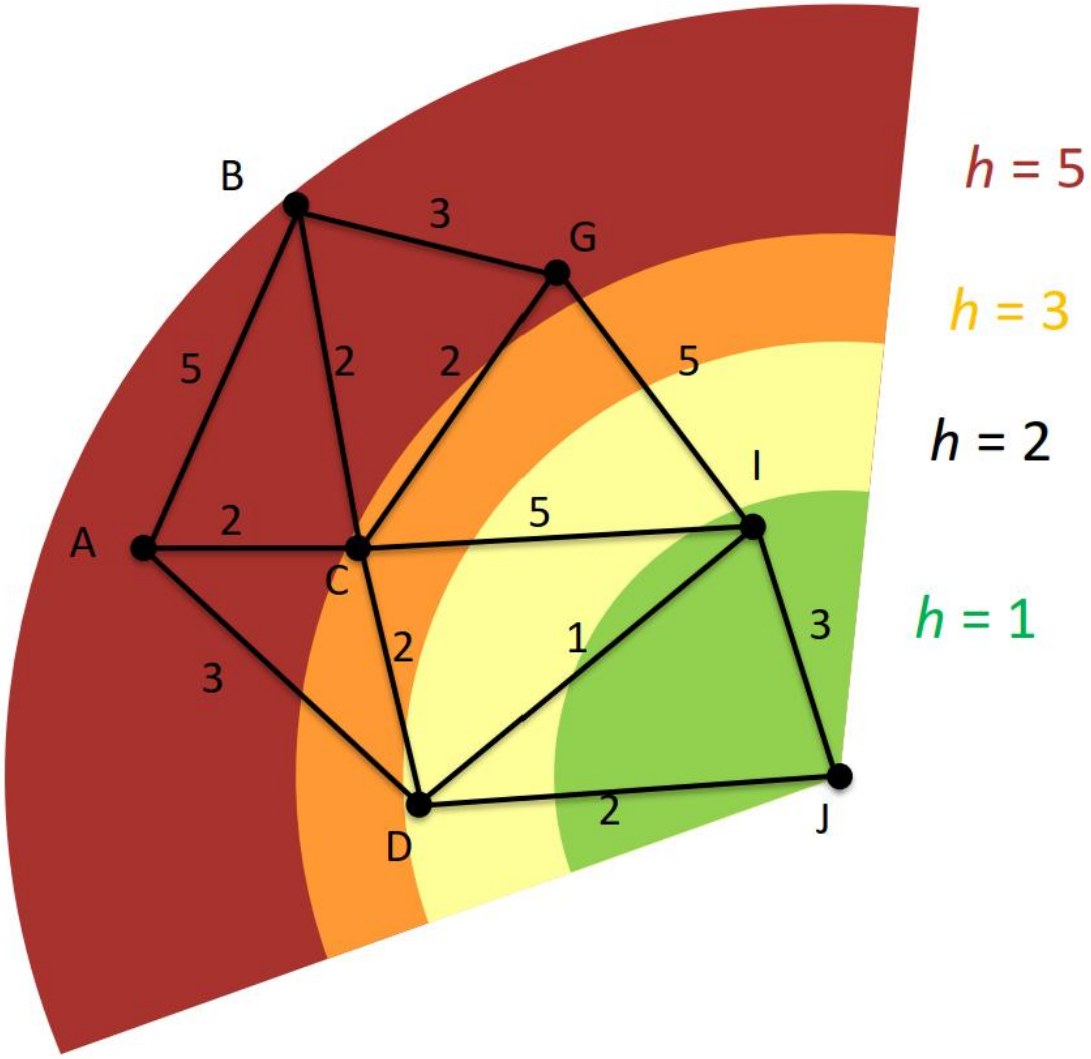
- A* Priority:

$$f(n) = g(n) + h(n)$$

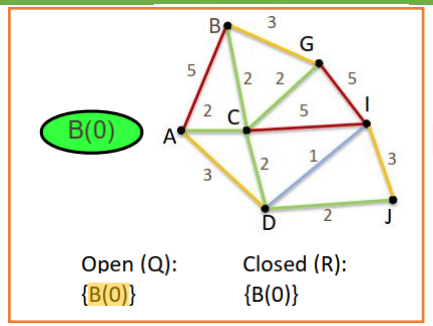
Cost to arrive

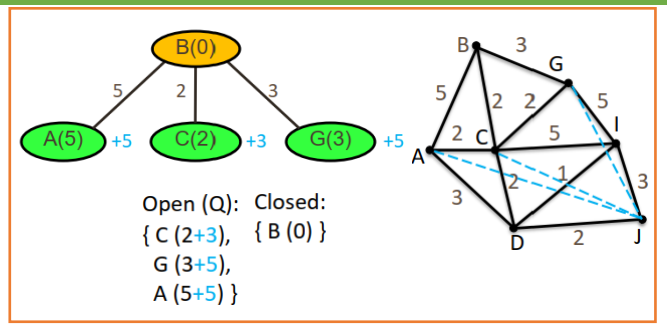
Heuristic cost to goal

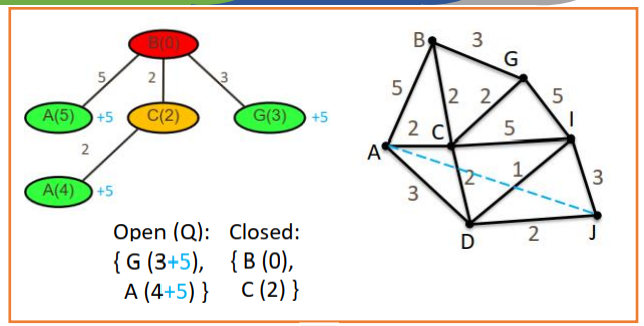
A* heuristic

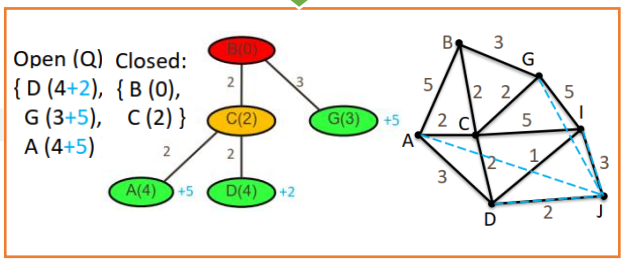


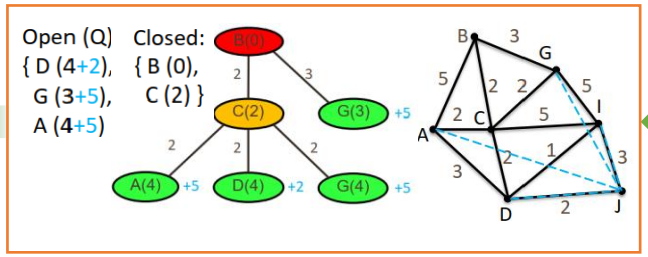
Example - A* algorithm

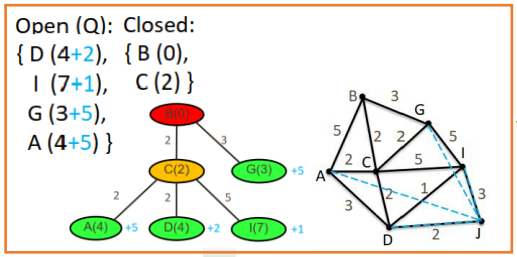


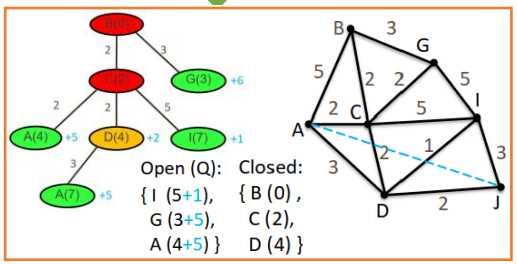


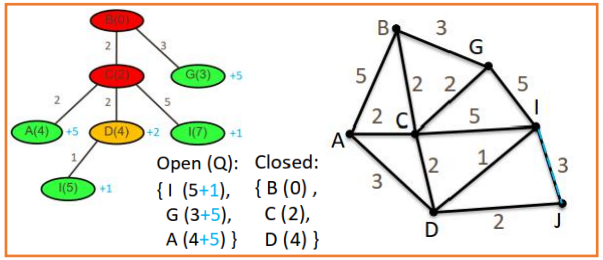


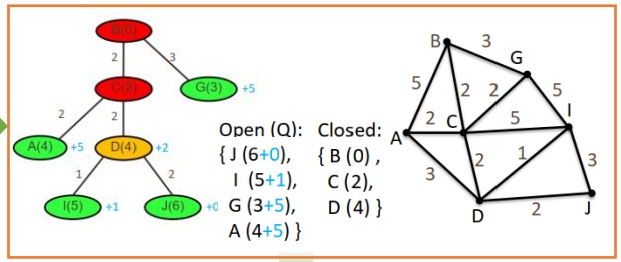


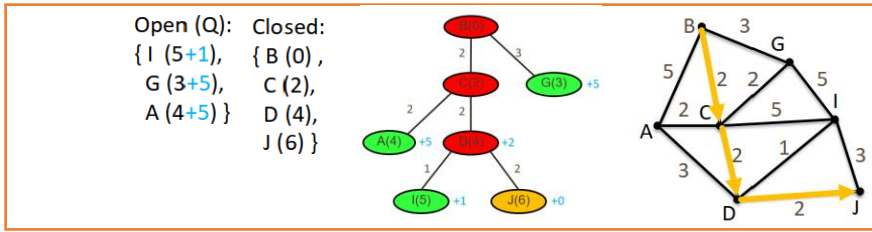












Final path solution: B→C → D → J
with path cost 6

A* heuristic

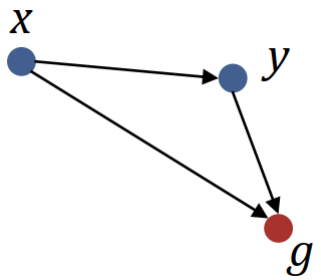
- The heuristic must be **admissible**:
  - It never overestimates the cost

True cost to goal

$$h(x) \leq d(x, goal)$$

- The heuristic must be **consistent**:
  - For any pair of adjacent nodes x and y, where *d*(*x*,*y*) is the cost of edge between them

$$h(x) \leq d(x, y) + h(y)$$

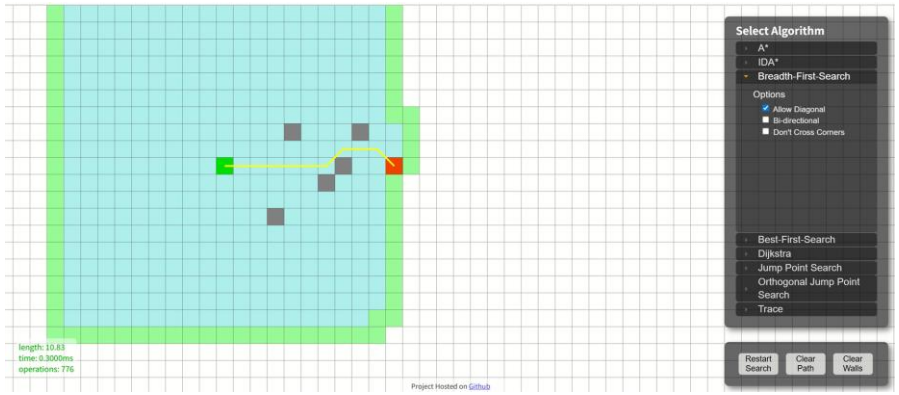


- Typical valid heuristics:
  - *Euclidean distance*
  - *Manhattan distance*
  - *Zero (Dijkstra's algorithm)*

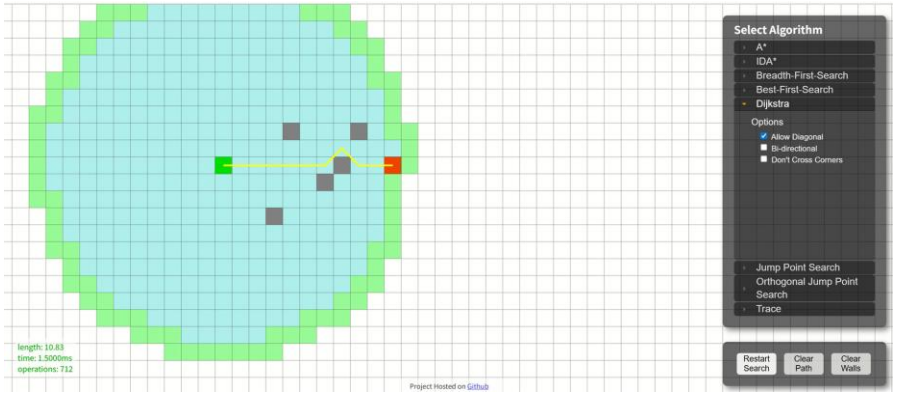
A* search algorithm

- A* is an extension of Dijkstra's algorithm, and achieves faster performance by using heuristics
- **Best-first search:** A* traverses a graph following a path of lowest expected total cost or distance
- The cost function is a sum of two functions:
  - Past path-cost function, which is a **known** cost from the starting node to the current node
  - Future path-cost function, which is a "heuristic estimate" of the distance from the current node to the goal

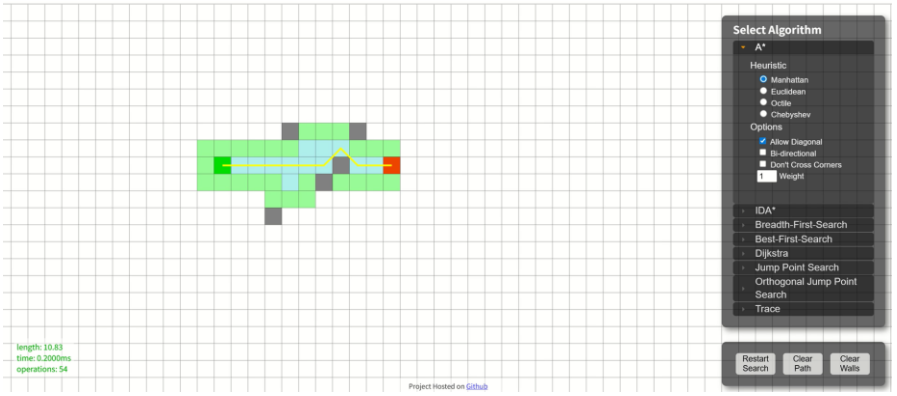
Demonstration



Breadth-first-search



Dijkstra's algorithm



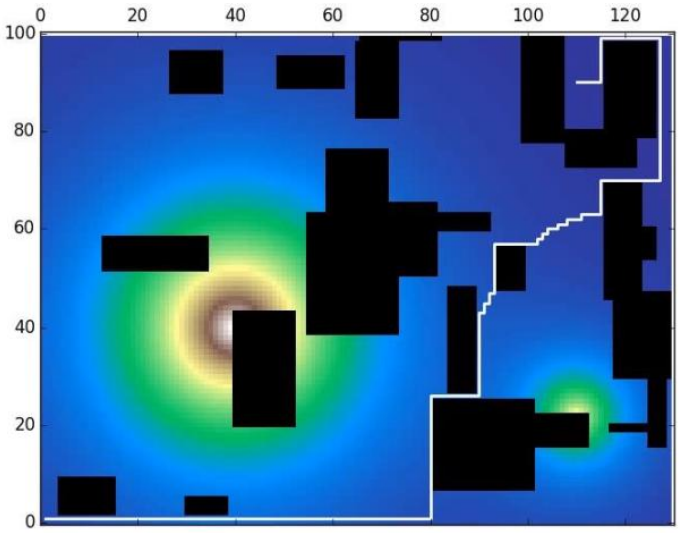
A* algorithm

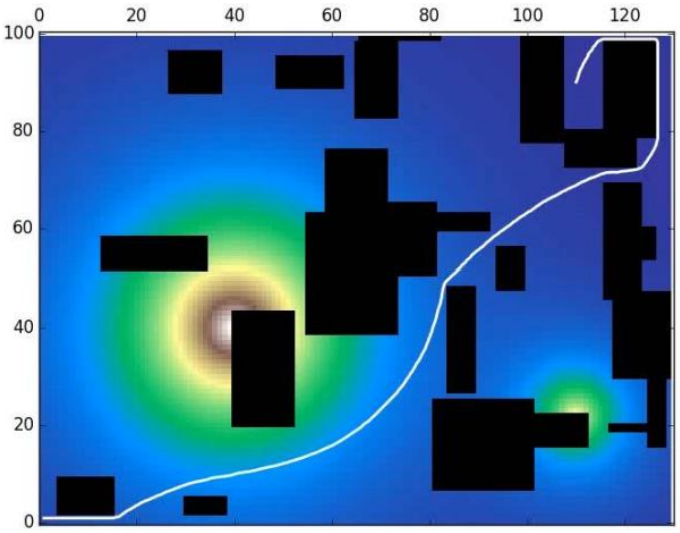
https://qiao.github.io/PathFinding.js/visual/

Limitations

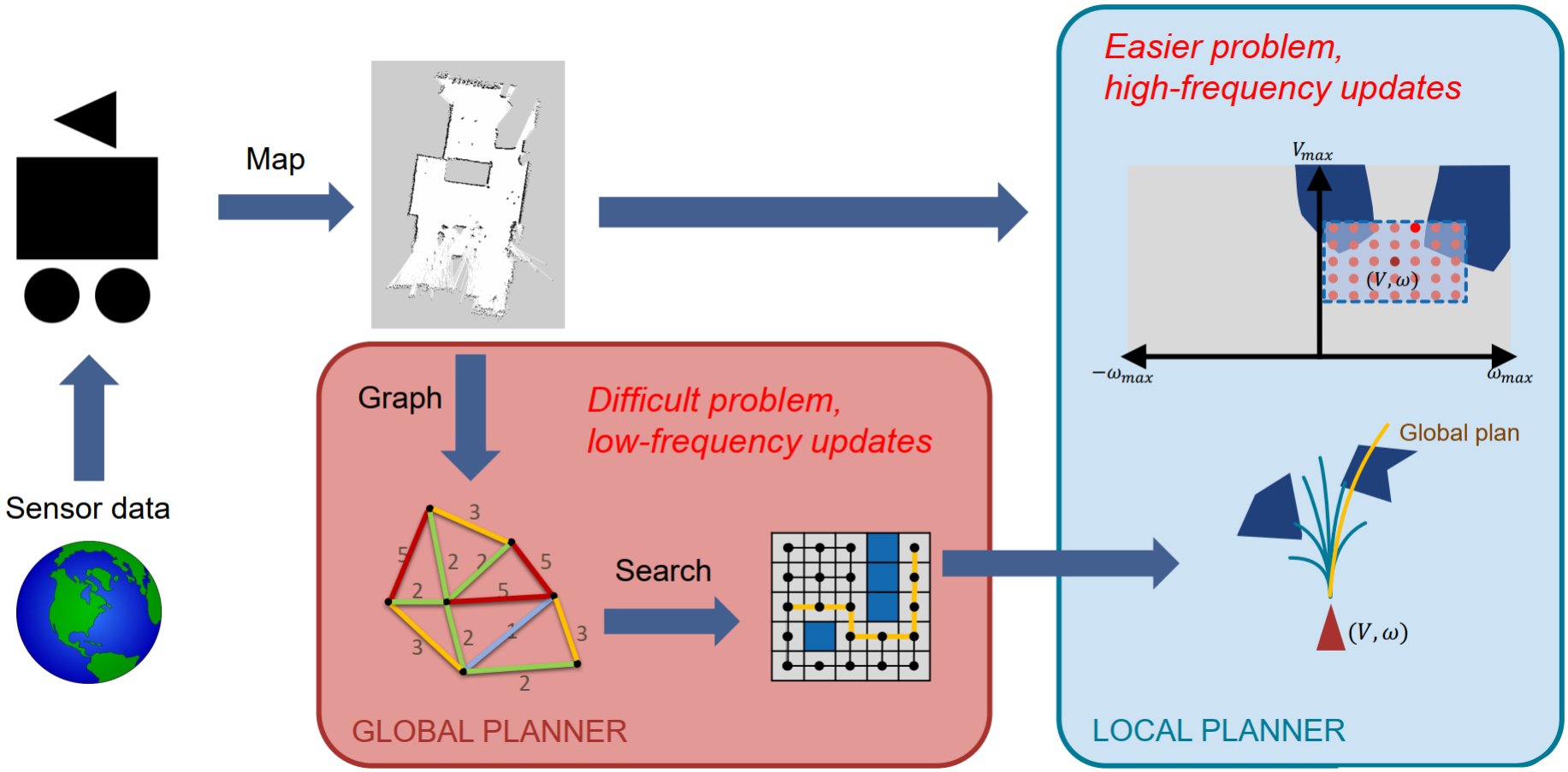
- A* is very commonly used in robot planning, especially for low-dimensional state spaces

- Limitations:
  - You need to construct a graph
  - Sometimes an admissible heuristic function is difficult to find (as hard as the problem)
  - A grid may not be a good representation of your problem



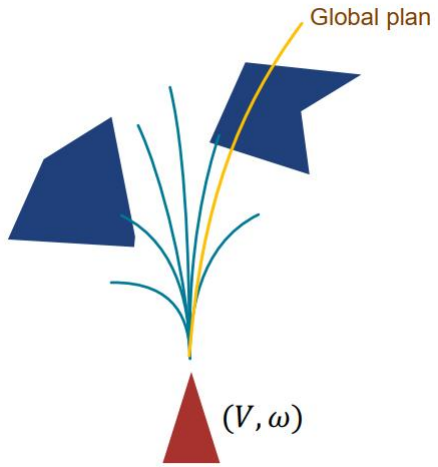


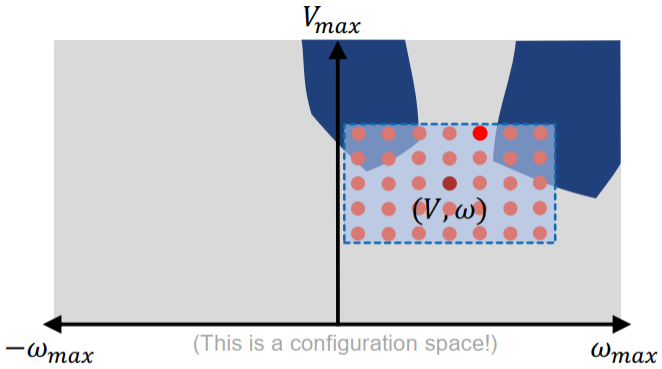
Planning hierarchy



Dynamic Window Collision Avoidance

- Generating full time-varying trajectories for $V(t)$ and $\omega(t)$ is still very challenging
- If we assume (V, ω) are constant for a fixed Δt, each local path in the future is a circular arc segment
- This can be easily considered as a type of velocity configuration space





- Maximise utility metric (usually maximise speed, minimise distance to goal, maximise distance from obstacles) across configuration samples

Fox, D., Burgard, W., & Thrun, S. (1997). The dynamic window approach to collision avoidance. IEEE Robotics & Automation Magazine, 4(1), 23-33.

Dynamic Window Collision Avoidance

- Velocity boundary constraints (V_m):

$$V_m = \{(v, \omega) | v \in [v_{min}, v_{max}], \omega \in [\omega_{min}, \omega_{max}]\}$$

- Acceleration constraints (V_d):

$$V_d = \{(v, \omega) | v \in [v_c - a_{vmax}\Delta t, v_c + a_{vmax}\Delta t], \omega \in [\omega_c - a_{\omega max}\Delta t, \omega_c + a_{\omega max}\Delta t]\}$$

- Safe constraints (V_a):

$$V_a = \{(v, \omega) | v \leq \sqrt{2dist(v, \omega)a_{vmax}}, \omega \leq \sqrt{2dist(v, \omega)a_{\omega max}}\}$$

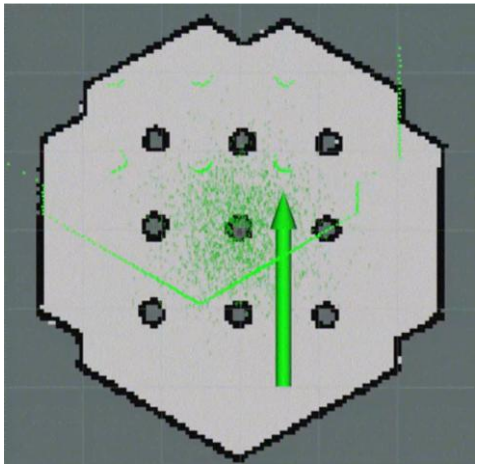
$dist(v, \omega)$ is the distance from the trajectory to the nearest obstacle at this velocity.

- Final dynamic window (V_r):

$$V_r = V_m \cap V_d \cap V_a$$

- Trajectory prediction:

$$\begin{cases} x_{k+1} = x_k + vcos(\theta_k)\Delta t \\ y_{k+1} = y_k + vsin(\theta_k)\Delta t \\ \theta_{k+1} = \theta_k + \omega\Delta t \end{cases}$$



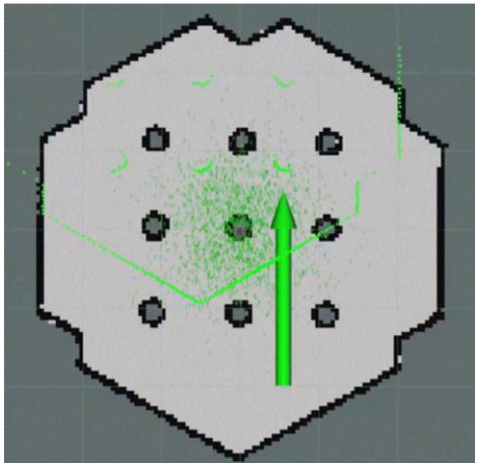
https://github.com/amslabtech/dwa_planner

Dynamic Window Collision Avoidance

- Evaluation function for trajectory candidate:

$$G(v,\omega) = \alpha \cdot heading(v,\omega) + \beta \cdot dist(v,\omega) + \gamma \cdot velocity(v,\omega)$$

  - $heading(v,\omega)$: Measures alignment with the goal. Typically, $heading(v,\omega) = 180° - |\theta_{\text{goal}} - \theta_{trajectory}|$
  - $dist(v,\omega)$: Represents clearance from obstacles. $dist(v,\omega) = \min_{t\in[0,\Delta T]}(distance_to_obstacle(t))$.
  - $velocity(v,\omega)$: Encourages forward progress. $velocity\big((v,\omega)\big) = |v|$



https://github.com/amslabtech/dwa_planner

- Regression:

$$heading(v,\omega)_{normal,i} = heading(v,\omega)_i / \sum_{i=1}^{n} heading(v,\omega)_i$$

$$dist(v,\omega)_{normal,i} = dist(v,\omega)_i / \sum_{i=1}^{n} dist(v,\omega)_i$$

$$velocity(v,\omega)_{normal,i} = velocity(v,\omega)_i / \sum_{i=1}^{n} velocity(v,\omega)_i$$

Example - DWA

Assume a mobile robot moves on a 2D plane. Its state is described by its position (x, y) and velocity (v_x, v_y). Given the state at the current time t:

$$x_t = 0\ \text{m}, y_t = 0\ \text{m}, v_x = 0.5\ \text{m/s}, v_y = 0\ \text{m/s}$$

The robot's maximum acceleration is $a_{\max} = 0.5\ \text{m/s}$, maximum velocity $v_{\max} = 1.0\ \text{m/s}$, velocity resolution $\Delta v = 0.2\ \text{m/s}$, and angular resolution $\Delta\theta = 45°$. The prediction horizon is $T = 2\ \text{s}$, and the simulation time step is $\Delta t = 0.5\ \text{s}$.

A set of velocity samples $(v_{x,c}, v_{y,c})$ is drawn from the dynamic window. Compute the predicted position after $T = 2\ \text{s}$ for one such sample. Complete the following tasks:

- (1) Based on the current velocity and acceleration limits, determine the velocity range (dynamic window) from *t* to *t* + *T*
- (2) Select one velocity sample from the dynamic window: $v_{x,c} = 0.8\ \text{m/s}$, $v_{y,c} = 0.2\ \text{m/s}$. Compute the robot's predicted position after applying this velocity for $2\ \text{s}$, assuming constant velocity motion during the prediction interval.

Example - DWA

Solution: **Step 1. Dynamic Window Velocity Range**

Current velocity $(v_x, v_y) = (0.5, 0)$ m/s. Maximum acceleration constraint: within T = 2 s, the velocity change cannot exceed $a_{\max} \times T = 0.5 * 2 = 1.0$ m/s

Therefore, the velocity bounds are

$$v_x \in [0.5 - 1.0, 0.5 + 1.0] = [-0.5, 1.5] \text{ m/s}$$
$$v_y \in [0 - 1.0, 0 + 1.0] = [-1.0, 1.0] \text{ m/s}$$

The maximum velocity constraint $|v_x| \leq v_{\max} = 1.0$ m/s must also be satisfied. Hence, the actual dynamic window is

$$(\hat{v}_x, \hat{v}_y) | -0.5 \leq \hat{v}_x \leq 1.0, -1.0 \leq \hat{v}_y \leq 1.0, \sqrt{\hat{v}_x{}^2 + \hat{v}_y{}^2} \leq 1.0\}$$

(Note: This range describes the theoretically reachable set; actual sampling discretizes it according to the given resolution.)

Example - DWA

Solution: **Step 2. Predicted Position Calculation**

Select a velocity sample: $v_{x,c} = 0.8\ \mathrm{m/s}$, $v_{y,c} = 0.2\ \mathrm{m/s}$ which is in the above window. Assuming constant-velocity motion during the prediction horizon *T*, the position after 2 s is:

$$x_{t+T} = x_t + \hat{v}_x \cdot T = 0 + 0.8 \times 2 = 1.6\ \mathrm{m},$$
$$y_{t+T} = y_t + \hat{v}_y \cdot T = 0 + 0.2 \times 2 = 0.4\ \mathrm{m},$$

Therefore, the predicted position is (1.6 m, 0.4 m).

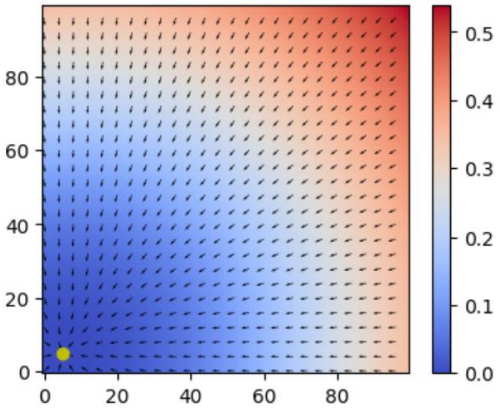


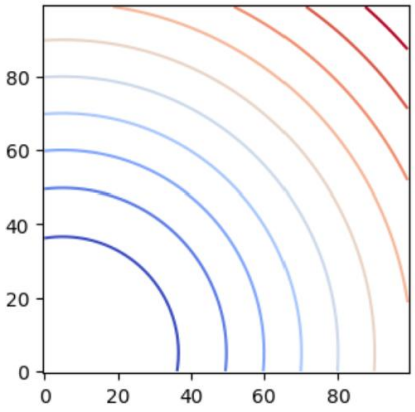
Note: In the actual DWA algorithm, the predicted trajectory accounts for piecewise acceleration integration, and multiple sampled velocities are evaluated (considering target heading, obstacle distance, speed magnitude, etc.). This exercise simplifies the computation by assuming constant velocity during the prediction interval.

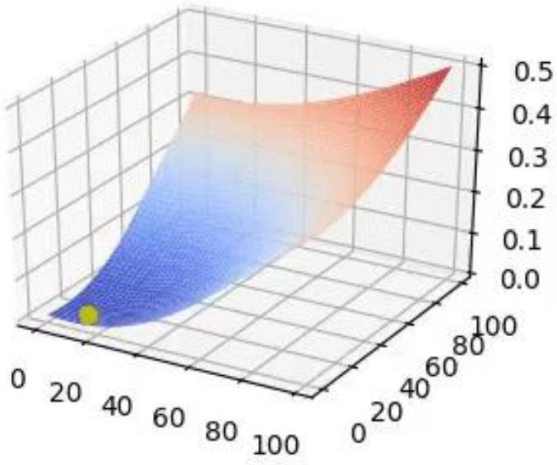
Potential Field methods – Global potential

- We want a smooth, differentiable function so that it is easy to calculate the target vector

$$U_{goal}(x) = \begin{cases} \frac{1}{2}\zeta \left\| x - x_{goal} \right\|^2, & \left\| x - x_{goal} \right\| < d^* \\ d^* \zeta \left(\left\| x - x_{goal} \right\|^2 - \frac{1}{2} d^* \right), & otherwise \end{cases}$$





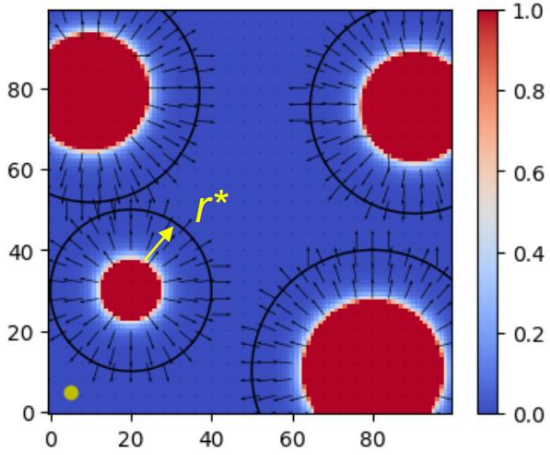


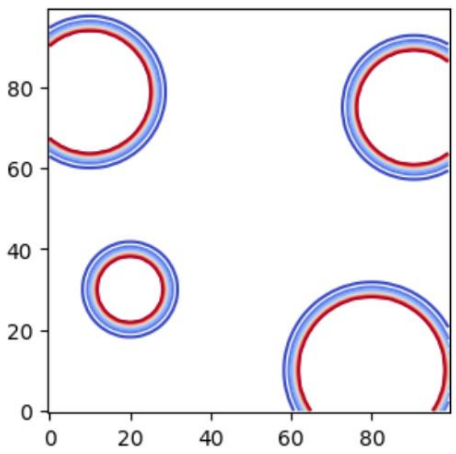
Potential Field methods – Obstacles

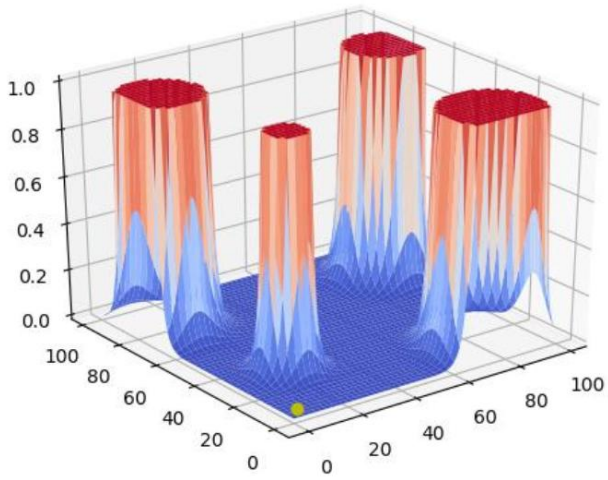
- We also want to avoid obstacles, so we add an additional component that 'repels' from obstacles

$$U_{obs}(x) = \begin{cases} \frac{1}{2}\eta\left(\frac{1}{D(x)} - \frac{1}{r^*}\right)^2, & D(x) \le r^* \\ 0, & otherwise \end{cases}$$

*$D(x)$ is the distance to the nearest obstacle boundary

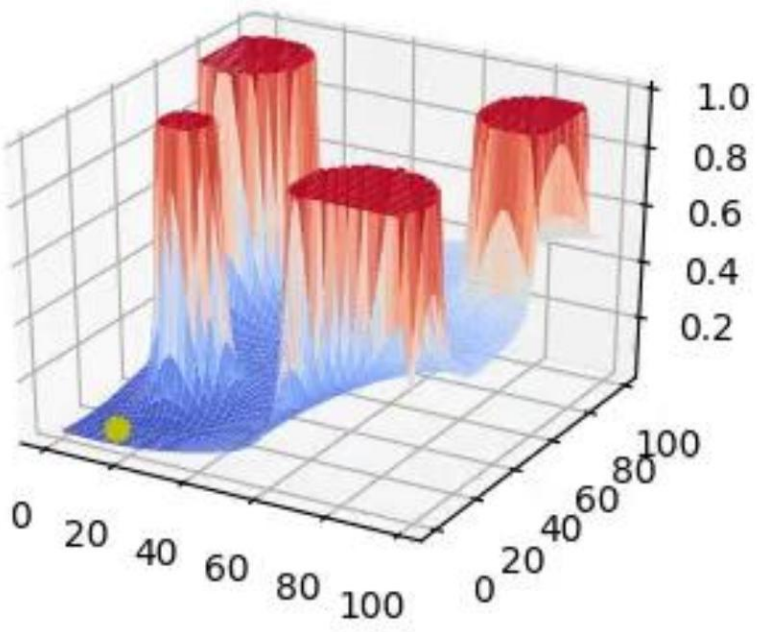


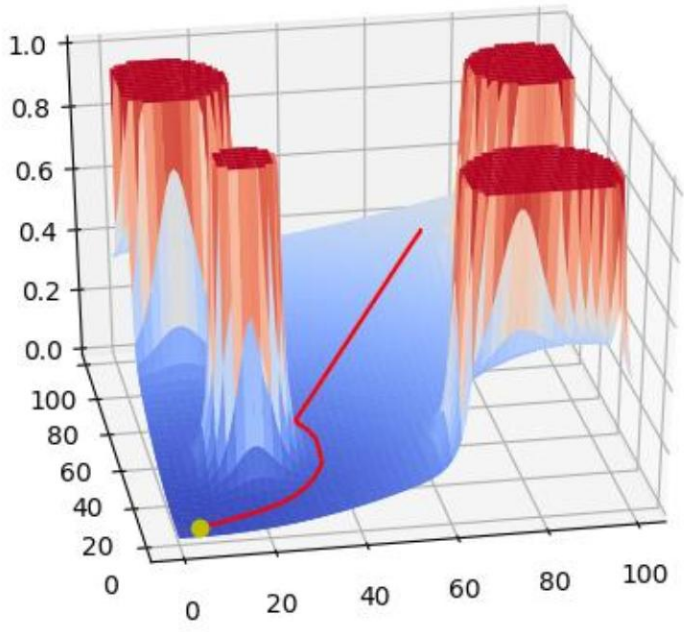




Potential Field methods

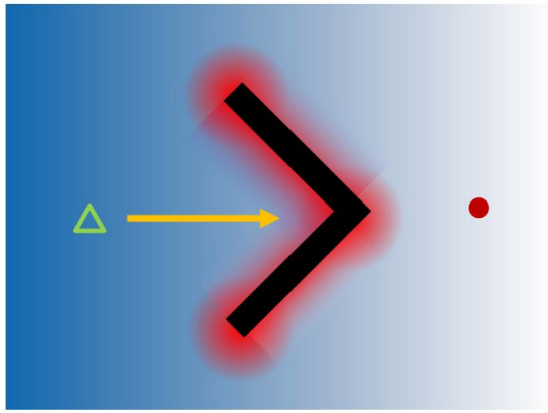
- The method generates an attractive potential function centered at the goal and local repulsive potentials around obstacles





Potential Field methods

- Relatively simple to implement
- Simplest versions can have issues with stationary points or local minima
- Modifying the potential functions (iterative solution) can allow these conditions to be avoided (harmonic potentials)
- Dealing with higher-order state spaces (arms etc.) can be difficult to grasp/visualize, but potential functions are applicable

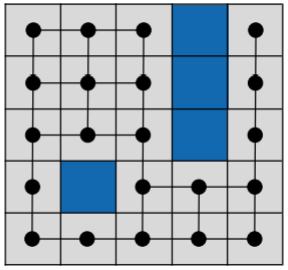


Sampling-based planning

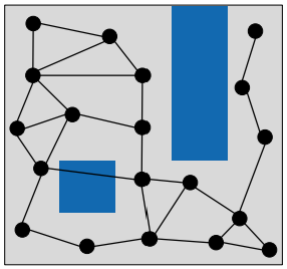
- To get around the problem of trying to divide the region uniformly into small cells, one approach is to randomly sample locations in the space and try and connect the samples

- Often, a larger proportion of the working volume is free space, so if two points are 'near' each other, it is often the case that they can be connected by a simple path

- **Basic idea of sampling-based planning:**
  - Abandon the concept of explicitly characterizing C_{free} and C_{obs}
  - A collision detection algorithm probes C to see whether some configuration lies in C_{free}



Explicitly use obstacles and grid to build valid graph

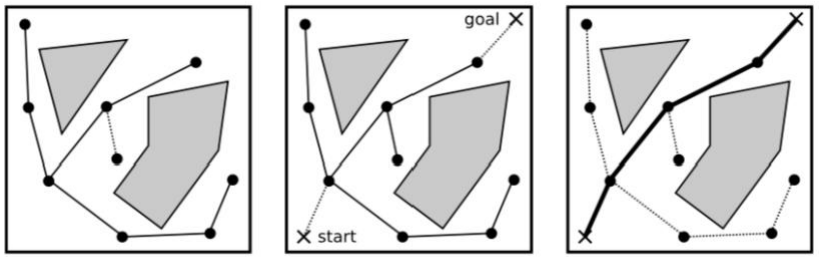


Use collision checks to build valid graph

Sampling-based planning

- Motivation for sampling-based:
  - Avoiding such a representation is the main underlying idea of sampling-based Methods
  - Instead of using an explicit representation of the environment, sampling-based methods rely on a collision-checking module

- The two most influential sampling-based motion planning algorithms to date are:
  - **Probabilistic Roadmaps** (PRM, 1996)
  - **Rapidly-exploring random trees** (RRT, 1998)

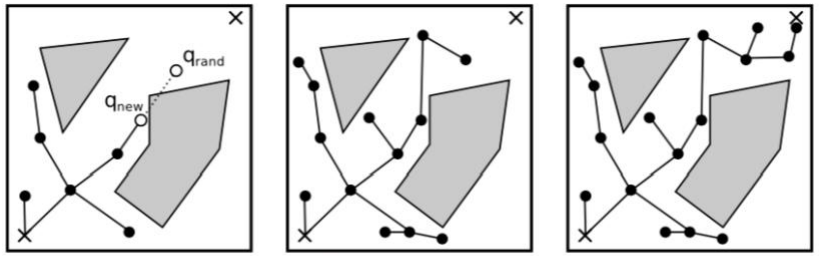


(b) A local planner is used to connect the new sample to nearby roadmap vertices.

(c) The *query* phase: the start and goal configurations are added to the roadmap.

(d) A graph search algorithm is used to connect the start and goal through the roadmap.

PRM algorithm



(b) The planner generates a configuration q_{rand}, and grows from the nearest node towards it to create q_{new}.

(c) The tree rapidly explores the free space.

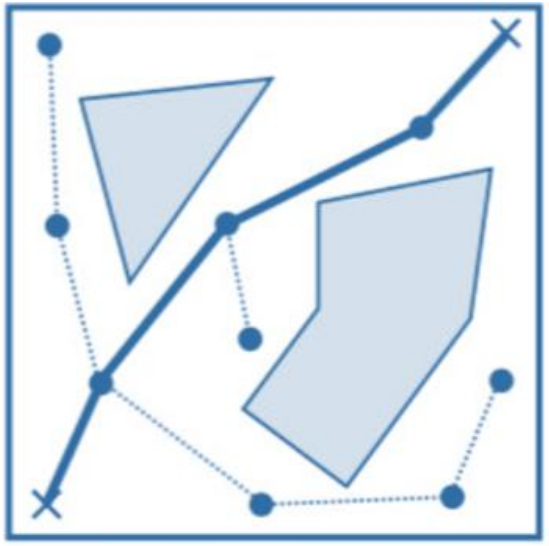
(d) The planner terminates when a node is close to the goal node. Common implementations will connect directly to the goal.

RRT algorithm

The general framework

- Incremental Sampling and Searching:

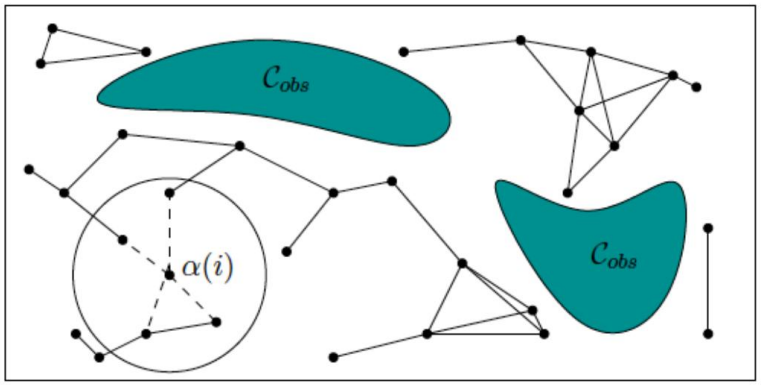
1. Initially, the graph is empty *G = (V, E)*
2. Then, a random configuration is generated and added to V
   - Need a **sampling algorithm**
3. For every new vertex c, select a number of nodes from V 'near' c and try to connect c to each of them using a local planner
   - Need to define a **distance function**
4. If local path is collision free, a new edge is added to *E*
   - Need **collision detection approach**
5. Check for a solution (connection between origin and target)
6. Return to step 2

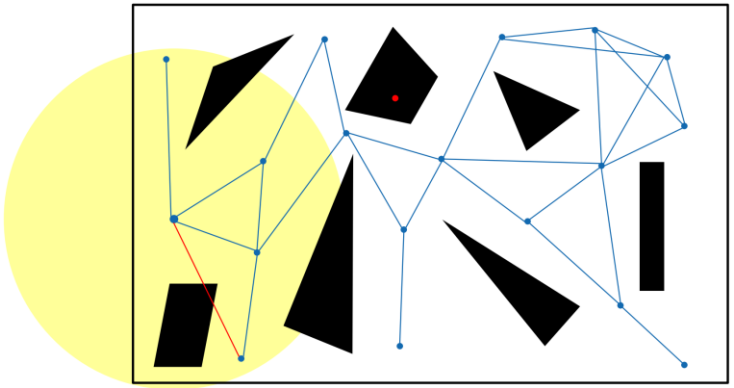


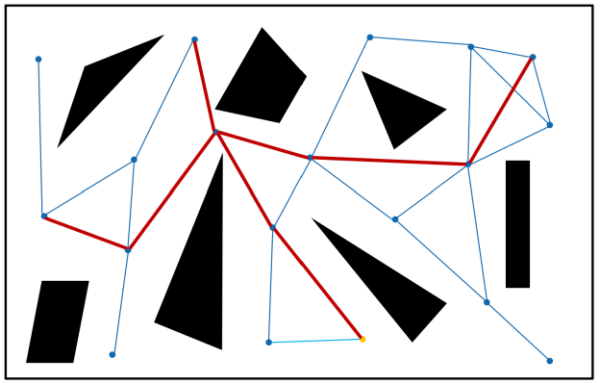
Probabilistic Road Maps

- Probabilistic Road Maps (PRMs):
  - *Step 1*: Build a roadmap by connecting nearby (sampled, free-space) configurations using simple planners to construct a graph of valid path segments
  - *Step 2*: Query; Search the graph using a graph search technique (A*)

```
BUILD_ROADMAP
1  G.init(); i ← 0;
2  while i < N
3      if α(i) ∈ C_free then
4          G.add_vertex(α(i)); i ← i + 1;
5          for each q ∈ NEIGHBORHOOD(α(i),G)
6              if ((not G.same_component(α(i), q)) and CONNECT(α(i), q)) then
7                  G.add_edge(α(i), q);
```





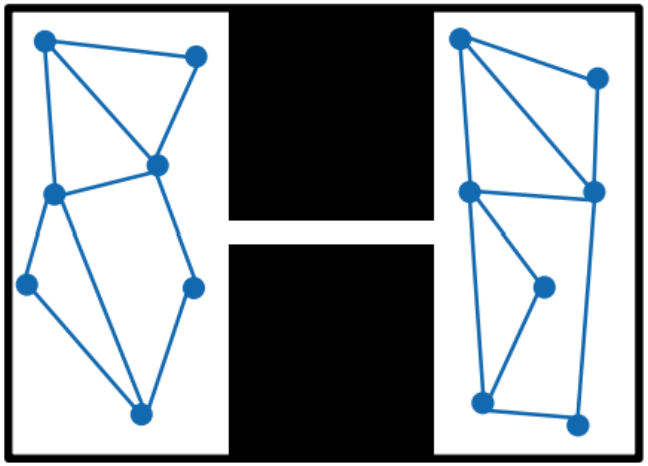


Probabilistic Road Maps

- Conceptually very simple, and became rapidly popular due to PRM's ability to solve previously challenging problems (high-dimension planning, snake robots, piano-mover):

- A few limitations with the original method, most of which were largely subsequently solved in derivative variations:
  - *Requires holonomic motion*
  - *Can suffer from 'narrow passage problem'*
  - *Not suited for dynamic environments*



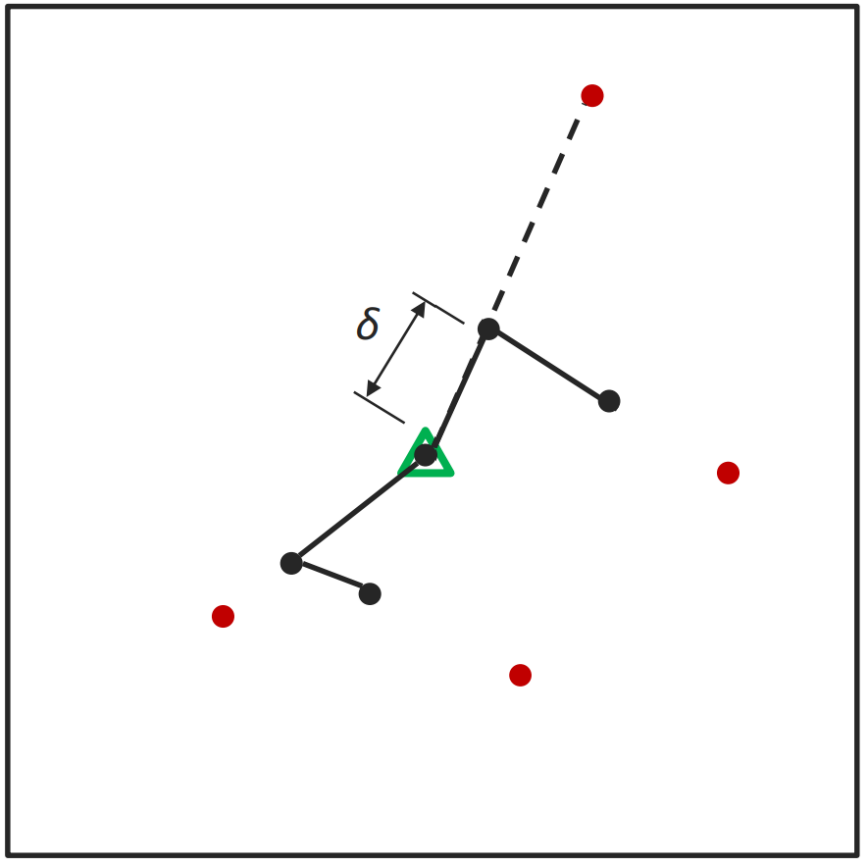


Rapidly-exploring Random Trees

- Problem – How do we connect all these disjoint samples?
- Solution – Instead of sampling all our target states at once, we start from a start configuration, and incrementally grow outwards from there – forming a tree of connected, dynamically feasible local paths
- Terminate when we reach a configuration in a goal region
- For single-query searches
- *Tree-based* vs *Roadmap-base* planners:
  - The tree-based is for single-query planning
  - In roadmaps, when planning with differential constraints, it's not easy to encode control information (undirected edges)
  - Tree-based, on the other hand, can include complex dynamics in their directed graphs (Control information can be encoded in each edge)

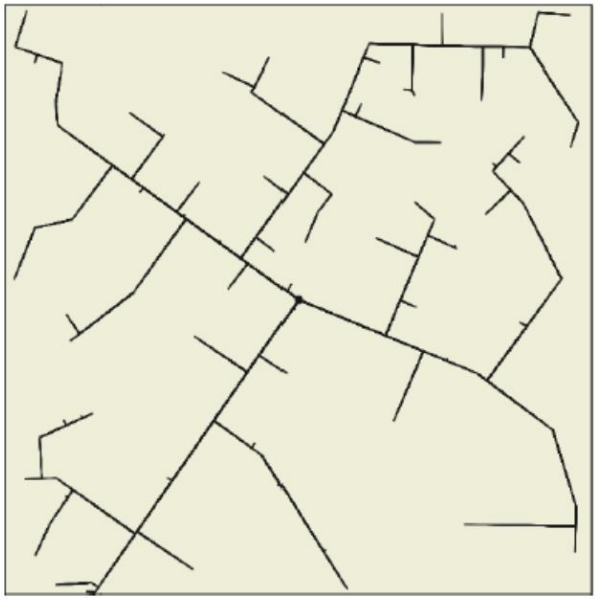
RRT - simple case

- Let's consider a simple case, where the motion is unconstrained (holonomic)

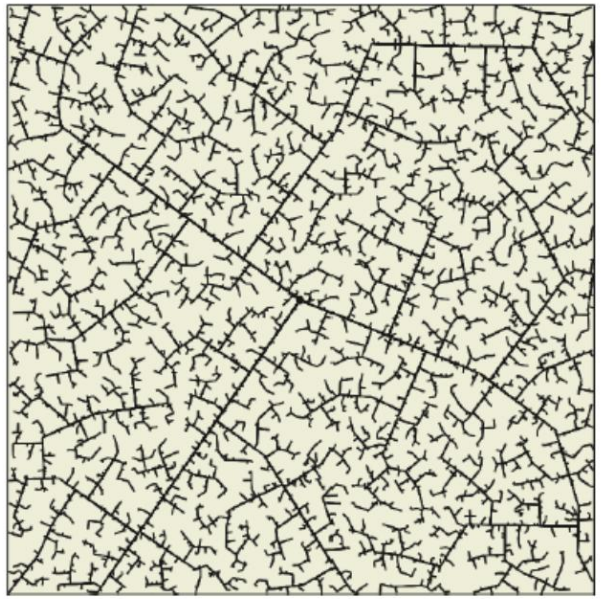


Rapidly-exploring Random Trees

- The main idea of Rapidly-exploring Random Trees (RRT) is to incrementally probe an explore the C-space:
  - In the early iterations the RRT quickly reaches the unexplored parts
  - However, RRT is dense in the limit, which means that it will eventually get to any point in the space



45 iterations



2345 iterations

Rapidly-exploring Random Trees

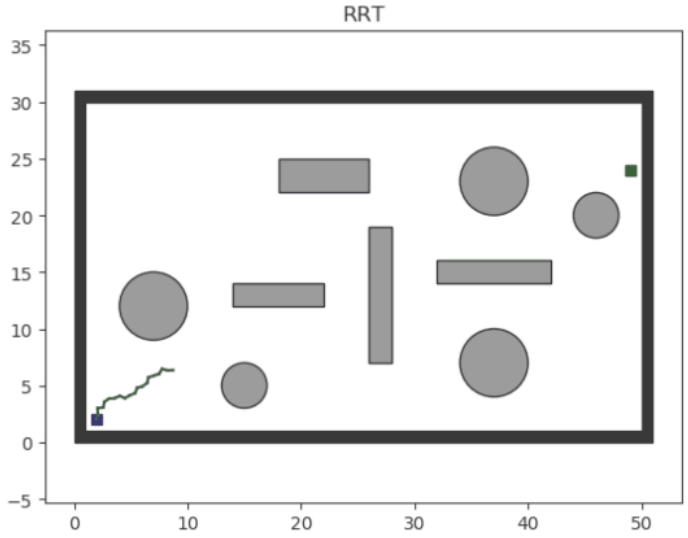
- RRT growing towards the goal

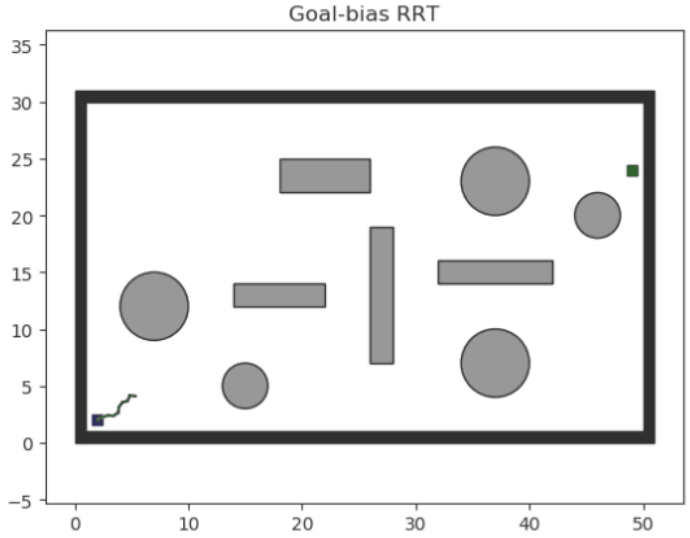
Input: q_{start}, q_{goal}, number n of nodes, stepsize α, β
Output: tree $T = (V, E)$

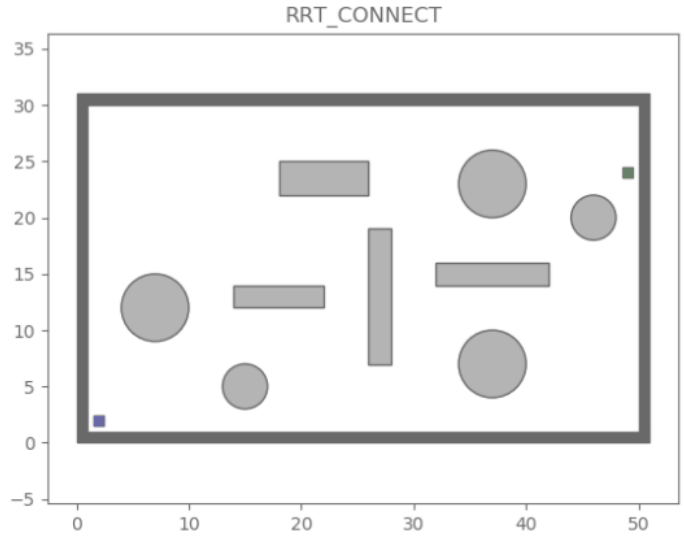
1: initialize $V = \{q_{\text{start}}\}$, $E = \emptyset$
2: **for** $i = 0 : n$ **do**
3: **if** rand$(0, 1) < \beta$ **then** $q_{\text{target}} \leftarrow q_{\text{goal}}$
4: **else** $q_{\text{target}} \leftarrow$ random sample from Q
5: $q_{\text{near}} \leftarrow$ nearest neighbor of q_{target} in V
6: $q_{\text{new}} \leftarrow q_{\text{near}} + \frac{\alpha}{|q_{\text{target}} - q_{\text{near}}|}(q_{\text{target}} - q_{\text{near}})$
7: **if** $q_{\text{new}} \in Q_{\text{free}}$ **then** $V \leftarrow V \cup \{q_{\text{new}}\}$, $E \leftarrow E \cup \{(q_{\text{near}}, q_{\text{new}})\}$
8: **end for**

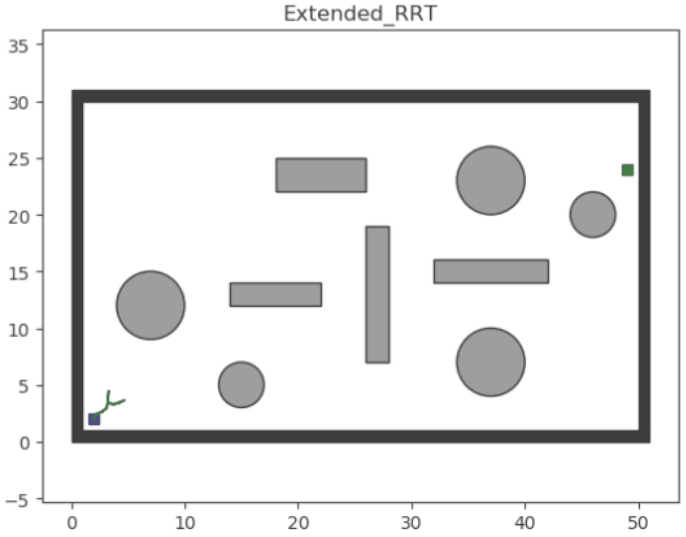
Rapidly-exploring Random Trees

- RRT & Variants https://github.com/zhm-real/PathPlanning



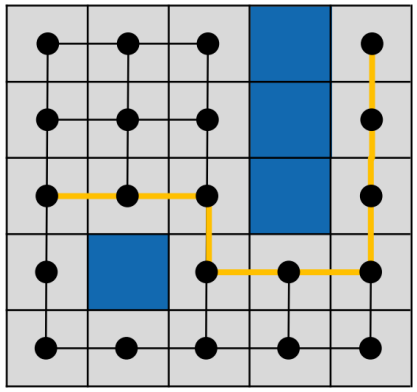


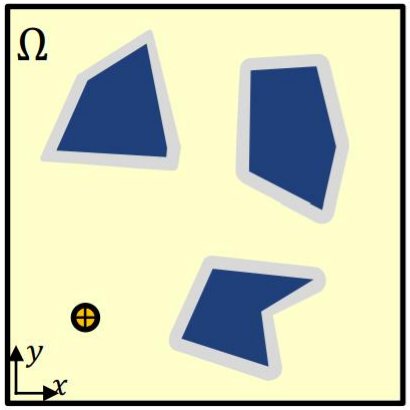


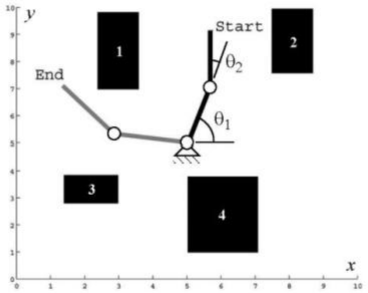


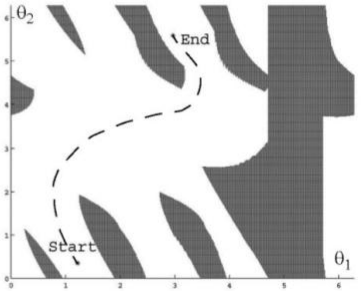
Representation summary

- Motion planning:
  - Representation – how to define the robot's understanding of the world, and ensure that it is sufficient to complete the task
  - Workspace – the world without the robot
  - Configuration space – the robot's configuration (joint angles etc.) in the world



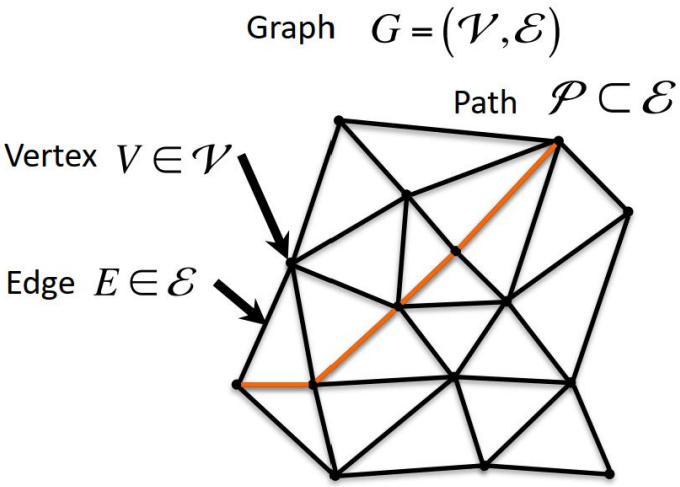






Graph search summary

- Graph search methods:
  - Graphs are constructions of vertices and connecting edges
  - Graph search techniques are used to find **low-cost paths** through graphs
  - **Breadth-first search (BFS)** and **depth-first search (DFS)** – complete searches from start (unweighted graphs)
  - **Djikstra** – search outwards in order of cost from start (weighted graphs)
  - **A*** – focused search that prioritises searching towards the goal using an admissible heuristic



Hierarchical planning summary

- Hierarchical planning:
  - Global planner over the whole search space
  - Local planner to respond to changes in environment, avoid collisions, stay on global path
- Dynamic window approach:
  - DWA is a velocity space-based, model-predictive local planner
- Potential fields:
  - Design a function such that descending the gradient leads to a collision-free path to the goal

PRM summary

- A graph is constructed in the configuration space by connecting random configurations
- Probabilistic Road Maps (PRM) is designed for multiple-queries
- The underlying assumption is that it is worth performing substantial precomputation on a given environment to enable multiple planning queries
- However, often we are interested only in an efficient single-query without precomputations:
  - *E.g. in dynamic environments, different robot models*
- In this case we want to combine exploration and search into a single method:

RRT summary

- Rapidly-exploring Random Trees (RRT) is designed for single-query searches
- Instead of sampling all our target states at once, we start from a start configuration, and incrementally grow outwards from there, forming a tree of connected, dynamically feasible local paths
- Terminate when we reach a configuration in a goal region

ROSE5730 Autonomous Mobile Robots

Lecture 6: Motion Planning

Dongkun Han
Department of Mechanical and Automation Engineering;
The Chinese University of Hong Kong.

